Can Power Draw Constrain Covert Compute? Limits of Analogue Verification for AI Governance

Anonymous Authors[1]

Abstract

Frontier AI treaties or agreements on limiting computation require external verification; an external auditor must be able to confirm how much computation actually ran and that parties are adhering to the agreement. Analogue, off-chip measurements such as power draw provide an information channel for verification. It is unknown how well these analogue channels can constrain the amount of computation, rather than the type of workload, against an adversary who actively tries to subvert the audit. We derive a closed form for β, the largest hidden computation a power trace cannot exclude. We measure β in units of the machine's full capacity in the declared numeric format, so β can exceed one when hidden work runs in a faster format. Measurements on NVIDIA A100 GPUs give $\beta = 1.16$ in the worst case, while an adversarial matched-energy schedule hides at least 0.41. Analogue power measurements alone therefore constrain compute weakly. Additional restrictions granted by the threat model, such as the ability of the verifier to re-execute the declared work at an observed operating point, let the verifier push β down to 0.059 in the maximally restricted case. This gives a quantitative estimate of what analogue measurements can contribute to compute verification.

1. Introduction

As frontier AI systems grow more capable (Sevilla et al., 2022; Bengio et al., 2026), governance proposals increasingly rely on *verification*: the ability of an outside party to check what a monitored operator is doing (Reuel et al., 2025; Baker et al., 2025). One proposed mechanism is a threshold on compute (Sastry et al., 2024). Above a specified number of floating-point operations (FLOPs), a training run triggers registration, reporting, or audit obligations (Shavit, 2023; Baker et al., 2025; Heim & Koessler, 2024). For example, U.S. Executive Order 14110 temporarily required reports on models trained using more than 10^{26} integer or floating-point operations (The White House, 2023; 2025).

Compute verification policies need to be robust to adversarial operators who do not behave honestly, but instead try to subvert the threshold by hiding computation. Verification methods that rely on passive telemetry such as utilisation counters, on-chip clocks, or memory traffic read silicon the operator controls. Consequently they rely on tamper-resistant read paths and attested monitoring firmware that do not yet exist (O'Gara et al., 2025; Ansari, 2026); even existing telemetry classifiers lose accuracy against disguised workloads (Rahman & Tajdari, 2026). Similarly, cryptographic proof-of-training protocols (Jia et al., 2021; Garg et al., 2023) depend on the operator's cooperation, while active challenge–response schemes (Monfared et al., 2026) run their probes on the operator's own untrusted hardware. Zero-knowledge proofs of inference (Sun et al., 2024) can show that a declared model produced an output. They cannot show how much computation that took: a "hollow" model with computationally trivial weights passes the same proof at a fraction of the cost (Gong et al., 2026). Proof-of-useful-work schemes, developed for blockchain consensus, prove a lower bound on the work done (Komargodski & Weinstein, 2025) and could help close that gap. At best, these methods bound the declared work from below. None of them bounds the undeclared work that can run beside it.

"Off-chip" analogue sensors have been suggested as an alternative verification method (Baker et al., 2025). This involves measurement hardware physically separate from the monitored chips such as a meter on the power feed, or an antenna, a microphone, or a thermal camera beside the rack. Each reads a physical side channel such as power, electromagnetic, acoustic, or thermal emissions and so its measurements reach the verifier without passing through any silicon or software the operator controls. Such sensors could be retrofitted to existing data centres and physically secured against tampering by the adversary.

There are reasons for optimism: in non-adversarial settings, physical emissions are known to leak workload informa-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Under review. Do not distribute.

tion. Physical side channels, such as a GPU's magnetic emanations (Maia et al., 2022) and snooped memory-bus traffic (Hu et al., 2020), let an observer recover model architectures and hyperparameters. Software-visible power and energy counters (Gao et al., 2024; Chaudhuri et al., 2025), and other counters and timing (Naghibijouybari et al., 2018; Wei et al., 2020), let a co-resident process do the same. On-node telemetry and on-chip counters classify workloads (Tang et al., 2022; Xu et al., 2026; Gangwal et al., 2020), and power draw betrays anomalous computation (Clark et al., 2013) and running GPU activity (Arefin & Serwadda, 2025). A related body of work characterises power draw for GPUs and data centres (Patel et al., 2023; Choukse et al., 2025; Latif et al., 2025). However, these are forward models, fitted largely in cooperative conditions, i.e. they map known workloads to their emissions. Most of this work also asks what *type* of workload ran. We ask a different question: *how much* computation ran, counted in operations. Verification poses the inverse problem (i.e. emissions to workload) against an operator actively trying to evade the sensor. It is an open question how well analogue-sensor measurements can constrain compute in that setting. This paper addresses that question for power draw measurements. The power channel is the channel most directly linked to computation, since every operation costs energy and total energy therefore budgets total work. We defer the study of other channels to future work.

This paper is organised as follows. Section 2 sets up the problem mathematically. Section 3 derives a conservative upper bound on the covert compute a power trace admits, in closed form, from a handful of physical bands. Section 4 empirically evaluates that bound by measuring the bands directly on NVIDIA A100 hardware. Section 5 shows how additional information gained by the verifier beyond the power trace (e.g. a precision declaration) pushes the bound lower. We close with a discussion in Section 6.

2. Setup and threat model

2.1. The forward model

Fix an observation window $t \in [0, T]$ and let $F(t)$ be the instantaneous rate of arithmetic operations, bounded by the hardware capacity $F_{\max}$. The forward model of $F(t)$ to power follows from the CMOS power law (Chandrakasan et al., 1992); chip power splits into a switching term and a static term,

$$P \;=\; \underbrace{\alpha\, C\, V^2 f}_{\text{dynamic}} + P_{\text{stat}}, \tag{1}$$

with V the supply voltage, f the clock frequency, C the total capacitance of the logic, and α the activity factor, i.e. the fraction of C that actually toggles in a given cycle ($0 \le \alpha \le 1$), so that αC is the capacitance switched per clock tick. A datapath that retires N_{op} nominal operations per cycle runs at operation rate $F = N_{\text{op}} f$. Substituting $f = F/N_{\text{op}}$ and collecting constants gives the forward model

$$P(t) \;=\; r(t)\, F(t) + P_0(t), \qquad r \;\equiv\; \alpha \Big(\frac{C}{N_{\text{op}}}\Big) V^2, \tag{2}$$

where P_0 collects P_{stat} and the remaining non-compute overhead (idle draw, cooling, I/O). We call r the *exchange rate* [J/op]. Note that the clock frequency has dropped out: what an operation costs does not depend on how fast the operations run, only on what each switching event costs and how many operations it delivers. Appendix A gives this derivation in full.

The exchange rate $r(t)$ is not a constant of the device but a property of the execution. The activity factor α depends on the operand values being multiplied (Chandrakasan et al., 1992). The capacitance switched per operation, C/N_{op}, depends on the numeric format (which sets both the datapath width and the packing N_{op}) and on where the operands reside (Horowitz, 2014). The supply voltage V tracks the voltage–frequency operating point (Pallipadi & Starikovskiy, 2006). Every one of them is under the prover's control, and none is visible to an off-chip meter. Empirically the spread is wide: component-level analyses of a 45 nm process put arithmetic and on-chip memory access at 0.03–100 pJ per operation depending on format and operand locality (Horowitz, 2014).

In practice, the meter reports only a noisy version of the power,

$$P_{\text{obs}}(t) = P(t) + \eta(t), \tag{3}$$

with $\eta(t)$ a zero-mean measurement noise the prover cannot forge. The noise is a measurement limit; zero-mean noise with a suitable correlation structure (stationary, finitely correlated) averages away over the window and shrinks the longer the verifier watches. Throughout this paper we therefore assume a perfect meter and work with the clean power $P(t)$.

We count hardware operations (i.e. what the silicon actually executes) rather than model FLOPs (i.e. the algorithmically required operations). Every executed operation costs joules, so the meter sees hardware operations. We count them as nominal operations of whichever numeric format runs. For example a multiply–add retires two nominal ops whether it executes at fp32 or int8, and integer multiply–adds are ops, not FLOPs.

2.2. The threat model

There are two parties. The *prover* is the monitored operator. It controls the workload, the operand values, the numeric formats, the clocks, and the idle states of the machine. It may wish to run compute it does not declare. The *verifier* is

the inspecting party: it reads P_{obs} from an off-chip meter it trusts, and it never sees r, F, or P_0 separately. The prover declares a workload of C_{dec} operations. The verifier wants to know if C_{dec} is consistent with P_{obs}. The verifier makes two assumptions:

Assumption 2.1 (Trusted meter)**.** The power trace cannot be forged: P_{obs} differs from the true draw P only by zero-mean noise, which we set to zero throughout.

Assumption 2.2 (Trusted declaration)**.** The declared computation really ran: the smallest total consistent with the window is C_{dec} itself. A power meter cannot by itself confirm this; independently checking that the declared work ran is a separate verification problem—the domain of re-execution, proof-of-learning, zero-knowledge proof and proof-of-useful-work protocols (Jia et al., 2021; Garg et al., 2023; Sun et al., 2024; Komargodski & Weinstein, 2025).

Everything else is under the prover's control and unobserved: the exchange rate $r(t)$, the overhead $P_0(t)$, and how the operation rate $F(t)$ splits between declared and covert work.

2.3. The inverse model

Define G as the forward map that carries a workload to its power trace,

$$G : \big(F(\cdot),\, r(\cdot),\, P_0(\cdot)\big) \;\longmapsto\; P(\cdot), \tag{4}$$

defined pointwise by Equation (2). The verifier faces an inverse problem: read the power trace and recover the workload that produced it, i.e. invert G. But G is non-injective. Since $r(\cdot)$ is free to vary, a low rate spent on much computation draws exactly the same power as a high rate spent on little. Distinct workloads (F, r, P_0) map to identical traces, and a many-to-one map has no inverse. Inversion therefore returns the identified set $\mathcal{A}(P_{\text{obs}})$ which collects every physically admissible workload that reproduces P_{obs} (Figure 1).

The question *"how many operations ran over the window?"* corresponds to one specific projection of $\mathcal{A}(P_{\text{obs}})$. Define the total compute

$$C \;=\; \int_0^T F\,dt \quad [\text{op}]. \tag{5}$$

Reading C off each admissible workload projects the identified set onto the C-axis,

$$\big\{\, C \;:\; (F, r, P_0) \in \mathcal{A}(P_{\text{obs}}) \,\big\} \;=\; [\,C_{\text{lo}},\, C_{\text{hi}}\,]. \tag{6}$$

We define the physical ambiguity β_{ID} as the width of that interval, measured against the machine's peak declared-format capacity $C_{\max} = F^{\text{dec}}_{\max} T$,

$$\beta_{\text{ID}} \;\equiv\; \frac{C_{\text{hi}} - C_{\text{lo}}}{C_{\max}}. \tag{7}$$

Under Assumption 2.2 the lower edge is the declaration itself, $C_{\text{lo}} = C_{\text{dec}}$. The interval runs from that honest baseline up to the most compute the meter cannot rule out, so the upper edge is the declared work plus the largest covert load a cheater could slip past the meter,

$$C_{\text{hi}} \;=\; C_{\text{dec}} \;+\; C^{\max}_{\text{cov}}. \tag{8}$$

Substituting $C_{\text{lo}} = C_{\text{dec}}$ and Equation (8) into Equation (7) leaves a single unknown,

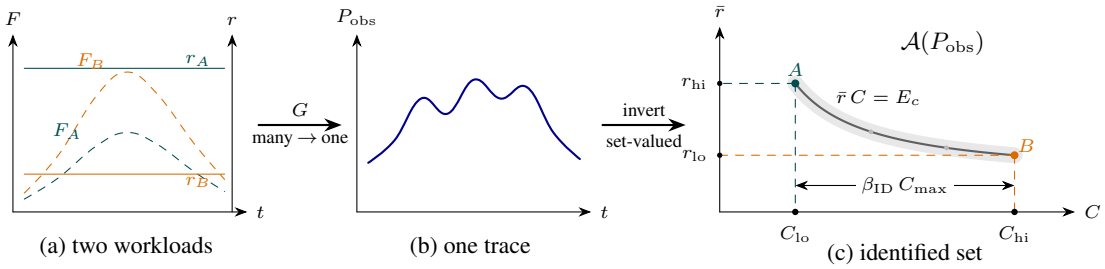
$$\beta_{\text{ID}} \;=\; \frac{C_{\text{hi}} - C_{\text{lo}}}{C_{\max}} \;=\; \frac{C^{\max}_{\text{cov}}}{C_{\max}}. \tag{9}$$

The width $C_{\text{hi}} - C_{\text{dec}} = C^{\max}_{\text{cov}}$ is exactly the covert operations a cheater could hide. A wider interval means a larger β_{ID} and more room for undeclared work. The unit $C_{\max}$ counts operations in the *declared* format. Covert work may run in a faster format, for example int8 against declared fp16 (Section 3.2), so β_{ID} can exceed one. A value above one means the meter cannot exclude more hidden work than the machine could do in a full window of declared-format operations.

To compute $C^{\max}_{\text{cov}}$ exactly, we would need the set of schedules that a real device can run. We do not know that set. Instead, we measure a *band* for each quantity the verifier cannot see: the range, from lowest to highest value, that the quantity takes in our experiments. There are three bands. They are the energy cost per operation of declared work, the energy cost per operation of covert work, and the overhead energy (Section 3). We then put each quantity at the edge of its band that helps the cheater most, independently of the other two. The resulting width is the *relaxed bound* β. Like β_{ID}, it is a single number in units of $C_{\max}$. It can only be wider: $\beta_{\text{ID}} \le \beta$. The two are equal only if one schedule can reach all three chosen edges at the same time (Appendix B). For a security claim, β errs in the safe direction. When we read β as a statement about what the hardware can actually schedule, the difference matters.

3. Derivation of β

We now derive a closed form for the relaxed bound β. Two physical constraints cap the covert work that can run inside the observation window $[0, T]$. The first is the *energy budget*: the total the meter reports must stay explainable as honest declared work, so covert operations can only spend joules that ambiguity leaves free (Section 3.1). The second is *capacity*: covert operations must also fit on the machine, in whatever machine-time the declared work leaves unused (Section 3.2). The bound β is the smaller of the two caps (Section 3.3).



(a) two workloads (b) one trace (c) identified set

Figure 1. Non-injectivity of the forward map. (a) Two workloads (A little compute at a high rate, B much compute at a low rate) whose products rF coincide, so (b) they drive the identical power trace. (c) Inverting recovers only the set of consistent explanations $\mathcal{A}(P_{\text{obs}})$: the rate band $[r_{\text{lo}}, r_{\text{hi}}]$ cuts the constant-energy arc A–B, whose projection onto the C-axis spans $[C_{\text{lo}}, C_{\text{hi}}]$; relative to capacity that width is the physical ambiguity β_{ID}. The faint grey ribbon is meter noise (ignored in this work). Each rate is drawn constant only for clarity; in reality $r(t)$ varies in time. This does not matter for a question about total operations, which depends on $r(t)$ only through its op-weighted mean $\bar{r} = E_c/C$ (compute energy $E_c = \int_0^T rF\,dt$ over total operations C). That mean is panel (c)'s vertical axis, so letting $r(t)$ vary leaves the picture unchanged.

3.1. The energy budget

The derivation of the first constraint is straightforward: integrate the forward model over the window, split the total energy into declared, covert, and overhead pieces, and ask how large the covert piece can grow before the total stops looking honest. We work with the integrated trace (i.e. the total energy) rather than the resolved time series: treating power as affine in the operation rate (Equation (2)) lets its integral depend on the workload only through time-integrated quantities.

Integrating the forward model Equation (2) over $[0, T]$, the left-hand side is the total energy the meter reports and the right-hand side falls into two pieces,

$$E \;=\; \underbrace{\int_0^T r(t)\,F(t)\,dt}_{\text{compute energy } E_c} \;+\; \underbrace{\int_0^T P_0(t)\,dt}_{\text{overhead } E_0}. \qquad (10)$$

The overhead E_0 is a nuisance the verifier knows only within a band $E_0 \in [E_{0,\text{lo}}, E_{0,\text{hi}}]$. We do not "subtract off" a known baseline: the prover controls fans, clocks, and idle states, so a known-and-removed P_0 would hand it a hiding channel.

The compute energy E_c splits into a declared block and a covert block,

$$\int_0^T r(t)\,F(t)\,dt \;= \underbrace{\int_0^T r^{\text{dec}}(t)\,F_{\text{dec}}(t)\,dt}_{\text{declared}} \;+\; \underbrace{\int_0^T r^{\text{cov}}(t)\,F_{\text{cov}}(t)\,dt}_{\text{covert}}. \qquad (11)$$

The declared block runs $C_{\text{dec}} = \int_0^T F_{\text{dec}}\,dt$ operations, taken known, and the covert block runs the excess $C_{\text{cov}} = C - C_{\text{dec}}$; this is the compute we are trying to bound. Write each block's energy as its operation count times a single mean cost per operation, i.e. the op-weighted averages

$$\bar{r}^{\,i} \equiv \frac{1}{C_i}\int_0^T r^i(t)\,F_i(t)\,dt \;=\; \int_0^T r^i(t)\,\frac{F_i(t)}{C_i}\,dt, \qquad (12)$$

for $i \in \{\text{dec}, \text{cov}\}$. The second form makes the averaging explicit: the weight $F_i(t)\,dt/C_i$ is the fraction of the block's operations that run in $[t, t+dt]$; it is nonnegative and integrates to one. The machine therefore draws

$$E \;=\; \underbrace{\bar{r}^{\text{dec}}\,C_{\text{dec}}}_{\text{declared work}} \;+\; \underbrace{\bar{r}^{\text{cov}}\,C_{\text{cov}}}_{\text{covert work}} \;+\; \underbrace{E_0}_{\text{overhead}}, \qquad (13)$$

and the verifier sees only E.

The two mean costs are bounded, $\bar{r}^{\text{dec}} \in [r^{\text{dec}}_{\text{lo}}, r^{\text{dec}}_{\text{hi}}]$ and $\bar{r}^{\text{cov}} \geq r^{\text{cov}}_{\text{lo}}$.[1] These band edges,

$$\left(r^{\text{dec}}_{\text{hi}},\; r^{\text{dec}}_{\text{lo}},\; r^{\text{cov}}_{\text{lo}}\right), \qquad (14)$$

are the whole parameter vector of this paper: from here on, all that changes is which setting of the triple a verifier can justify.

The verifier raises no alarm so long as the measured total is explainable by honest declared work at some admissible rate and overhead. The most an honest run could ever draw is the declared work at the top of its band, on top of the highest overhead,

$$E \;\leq\; r^{\text{dec}}_{\text{hi}}\,C_{\text{dec}} \;+\; E_{0,\text{hi}}. \qquad (15)$$

The test is one-sided because covert work can only add energy: only a total that runs high is suspicious. Substituting the budget Equation (13) into the no-alarm condition

[1] No upper edge is needed for the covert cost: $\bar{r}^{\text{cov}}$ enters the bound only as a divisor in Equation (16), so the worst case sits at its lower edge. Upper limits are uninformative as expensive covert work only hurts the adversary.

Equation (15) and rearranging,

$$C_{\mathrm{cov}} \;\le\; \frac{\left(r_{\mathrm{hi}}^{\mathrm{dec}} - \bar{r}^{\mathrm{dec}}\right) C_{\mathrm{dec}} \;+\; \left(E_{0,\mathrm{hi}} - E_0\right)}{\bar{r}^{\mathrm{cov}}}. \qquad (16)$$

A cheater now pushes every free choice on the right-hand side to its limit:

- *Run the declared work cheaply,* $\bar{r}^{\mathrm{dec}} \to r_{\mathrm{lo}}^{\mathrm{dec}}$. The verifier cannot tell an expensive declared run from a cheap one, so it must allow the draw all the way up to $r_{\mathrm{hi}}^{\mathrm{dec}}\, C_{\mathrm{dec}}$; the gap $(r_{\mathrm{hi}}^{\mathrm{dec}} - r_{\mathrm{lo}}^{\mathrm{dec}})\, C_{\mathrm{dec}}$ is free energy the prover spends covertly while the total still reads as honest work.
- *Idle the overhead,* $E_0 \to E_{0,\mathrm{lo}}$, freeing the full band width $\Delta E_0 = E_{0,\mathrm{hi}} - E_{0,\mathrm{lo}}$.
- *Run the covert work cheaply,* $\bar{r}^{\mathrm{cov}} \to r_{\mathrm{lo}}^{\mathrm{cov}}$, so each hidden operation costs as little as the hardware allows.

At those values Equation (16) attains its maximum, the largest covert load the energy budget admits,

$$C_{\mathrm{cov}}^{\max,\,\mathrm{en}} \;=\; \frac{\left(r_{\mathrm{hi}}^{\mathrm{dec}} - r_{\mathrm{lo}}^{\mathrm{dec}}\right) C_{\mathrm{dec}} \;+\; \Delta E_0}{r_{\mathrm{lo}}^{\mathrm{cov}}}. \qquad (17)$$

Divide by $C_{\max}$ and write the declared hardware utilisation $h = C_{\mathrm{dec}}/C_{\max}$ for the fraction of the machine's declared-format capacity the declared work occupies. The result is a covert load in the normalised units of Equation (7): the relaxed bound if energy were the only constraint. We write it β_{en}, the *energy-only width*,

$$\beta_{\mathrm{en}}(h) \;=\; h\,\frac{r_{\mathrm{hi}}^{\mathrm{dec}} - r_{\mathrm{lo}}^{\mathrm{dec}}}{r_{\mathrm{lo}}^{\mathrm{cov}}} \;+\; \frac{\Delta E_0}{r_{\mathrm{lo}}^{\mathrm{cov}}\, C_{\max}}. \qquad (18)$$

Pedagogically, Equation (18) describes (how much honest work there is) × (how uncertain its per-operation cost is), divided by (how cheaply covert work can run), plus a smaller slack from the overhead band. The first term grows with the declared-band width and with the declared utilisation: more legitimate work offers more cover. The second term is the overhead's contribution: joules the prover frees by idling overhead that the verifier must still allow for.

Note the asymmetry between the numerator and denominator of Equation (18). Declarations and re-execution can narrow the declared band width $r_{\mathrm{hi}}^{\mathrm{dec}} - r_{\mathrm{lo}}^{\mathrm{dec}}$ in the numerator, but no declaration reaches the covert floor $r_{\mathrm{lo}}^{\mathrm{cov}}$ in the denominator; only hardware-wide observables (clock, voltage) can raise $r_{\mathrm{lo}}^{\mathrm{cov}}$. The overhead width ΔE_0 is a second numerator, and no declaration reaches it either: it is freedom the prover exercises inside the window rather than ignorance about a quantity fixed during it, so characterising it off-site measures it without shrinking it. It yields only to an on-site observable of the idle state, or to a restriction on which overhead states may accompany declared work (Section 5).

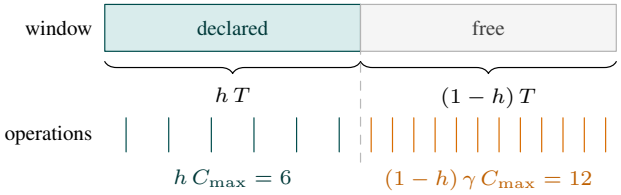


Figure 2. The capacity clip. Declared work takes $h\,T$ of the window and the covert block gets the rest (top); each block executes operations at its own format's rate (bottom), drawn at $h = 1/2$ and $\gamma = 2$ (int8 covert work against declared fp16). The free half-window hides a full $C_{\max} = 12$ worth of covert work.

3.2. The capacity clip

The second constraint is capacity: covert work must also fit on the machine. The declared and covert streams multiplex one device across a single window of length T, so the covert block runs only in the time the declared block leaves free. What turns free time into an operation count is a rate. The realised rate $F(t)$ is the prover's to choose, so a worst-case bound must price the time at the one rate the verifier does know: the published peak $F_{\max}$. But the peak is not just a property of the hardware; it is also a property of the executing format. An A100's dense peaks run from 19.5 Top/s at fp32 on the CUDA cores through 156 at tf32 and 312 at fp16/bf16 to 624 Top/s at int8 on the tensor cores (NVIDIA Corporation, 2020). We define the capacity ratio of the covert format against the declared one,

$$\gamma \;=\; \frac{F_{\max}^{\mathrm{cov}}}{F_{\max}^{\mathrm{dec}}} \qquad (19)$$

($\gamma = 2$ for int8 covert work against fp16 declared work on an A100). The declared work of $C_{\mathrm{dec}} = h\,C_{\max}$ operations cannot be executed in less than $C_{\mathrm{dec}}/F_{\max}^{\mathrm{dec}} = h\,T$ of the window, so at most $(1-h)\,T$ is left free (Figure 2). Run at the covert format's peak, that free time is worth $F_{\max}^{\mathrm{cov}}\,(1-h)\,T$ covert operations. Dividing by $C_{\max} = F_{\max}^{\mathrm{dec}}\,T$ gives

$$C_{\mathrm{cov}}/C_{\max} \;\le\; (1-h)\,\gamma. \qquad (20)$$

When $\gamma > 1$ the covert format executes more operations per unit of machine-time than the declared one, so $C_{\mathrm{dec}} + C_{\mathrm{cov}}$ may exceed $C_{\max}$: the normaliser is the machine's capacity priced in declared-format operations, not a ceiling on how many operations can run.

3.3. The relaxed bound

The covert load must fit under both relaxed caps at once, so the bound is the smaller of the two,

$$\beta(h) \;=\; \min\left[\,\beta_{\mathrm{en}}(h),\; (1-h)\,\gamma\,\right]. \qquad (21)$$

The two caps pull in opposite directions (Figure 3). Energy rises with h—more declared work offers more cover—while

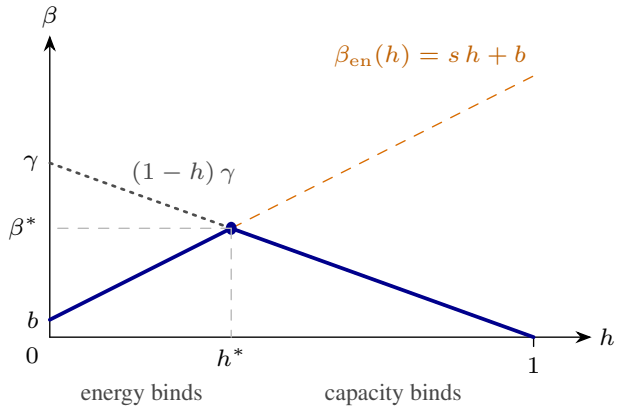


Figure 3. The shape of the relaxed bound. The energy-only width $\beta_{\mathrm{en}}(h) = s\,h + b$ rises with declared utilisation—more honest work offers more cover (dashed)—while the capacity clip $(1-h)\,\gamma$ falls, since more honest work leaves less machine free (dotted). The bound Equation (21) is the lower of the two (solid): a tent peaking at the crossing (h^*, β^*) of Equation (22), with energy binding to its left, capacity to its right, and $\beta(1) = 0$ because a fully occupied machine leaves nowhere to hide.

capacity falls—more declared work leaves less machine to hide on. At full declared utilisation the machine is spoken for and $\beta(1) = 0$; at idle there is capacity but almost no cover. The declared utilisation is the prover's to set, so the guarantee a verifier can quote is the worst case over it, $\beta^* \equiv \max_h \beta(h)$. The corresponding physical quantity is $\beta^*_{\mathrm{ID}} \equiv \max_h \beta_{\mathrm{ID}}(h)$, and the relaxation guarantees $\beta^*_{\mathrm{ID}} \le \beta^*$. The paper tracks the computable bound β^*. Its maximum sits at the crossing,

$$h^* = \frac{\gamma - b}{s + \gamma}, \qquad \beta^* = \gamma\,\frac{s + b}{s + \gamma}, \tag{22}$$

writing $\beta_{\mathrm{en}}(h) = s\,h + b$ for its slope and intercept. The crossing form holds when $b < \gamma$; if the overhead term alone exceeds the clip ($b \ge \gamma$) the clip binds everywhere and the maximum sits at the boundary, $(h^*, \beta^*) = (0, \gamma)$. Each side of the minimum is the binding constraint in its own regime: energy binds for $h < h^*$ (the trace itself caps the covert load), capacity binds for $h > h^*$, and both bind at the crossing. Where the crossing sits is an empirical question about s.

The relaxation is exact only if one schedule can attain the three selected edges together—cheap declared work, low overhead, and cheap covert work. No such joint attainment is assumed, so throughout $\beta_{\mathrm{ID}}(h) \le \beta(h)$. Appendix B discusses this joint-attainment assumption and brackets the remaining gap with a constructed schedule.

4. How big is β?

We now estimate the typical value of β, Equation (21), for real hardware. We first ask what physically drives each entry of the triple Equation (14) (Section 4.1), then measure them directly on an NVIDIA A100, where each experiment is built to pin one entry (Section 4.2). We focus on the $\beta_{\mathrm{en}}(h)$ component, Equation (18): it carries all the physics, whereas the clip Equation (20) is a ratio of published peaks with nothing to measure. The capacity clip Equation (20) returns at the end, when the measured bands are assembled.

4.1. The two terms

The energy-only width $\beta_{\mathrm{en}}(h)$ is a sum of two terms: an ambiguity term $s\,h$ with slope $s = (r^{\mathrm{dec}}_{\mathrm{hi}} - r^{\mathrm{dec}}_{\mathrm{lo}})/r^{\mathrm{cov}}_{\mathrm{lo}}$, set by the spread of the declared exchange rate against the price of a covert operation, and an overhead term $b = \Delta E_0/(r^{\mathrm{cov}}_{\mathrm{lo}}\,C_{\max})$, set by the width of the overhead band against the same price. We take each term in turn.

The exchange-rate term. Section 2.1 wrote the exchange rate as $r = \alpha\,(C/N_{\mathrm{op}})\,V^2$. Here we split the capacitance term in two, since the numeric format and the operand locality move independently:

$$r \;\approx\; \underbrace{\kappa}_{\text{precision}} \cdot \underbrace{\alpha}_{\text{operands}} \cdot \underbrace{C_{\mathrm{eff}}}_{\text{locality, kernel}} \cdot \underbrace{V^2}_{\text{DVFS}}. \tag{23}$$

Appendix A derives this factorisation. Each factor is set by a different physical scale of the machine (Figure 4). The numeric-format scale κ decreases with datapath width at each precision step (fp32 → int8), tracking the pJ-scale energy hierarchy of arithmetic (Horowitz, 2014); the activity factor α is the data-dependent fraction of transistors that toggle for the given operands; the effective capacitance C_{eff} grows as operands become less local, because more of the memory hierarchy (e.g. storage arrays and peripheral logic, buffers and interfaces, interconnect) switches per operation (Horowitz, 2014; Chatterjee et al., 2017); and V^2 is the capacitor-charging energy of the supply voltage, which DVFS slides along the voltage–frequency curve (Pallipadi & Starikovskiy, 2006).

With the power meter as the only channel, every factor of Equation (23) is conceded at once. If each ranges over an interval the verifier cannot narrow, and the factors can be extreme together, the declared band's edges are the products of the factor edges,

$$r^{\mathrm{dec}}_{\mathrm{hi}} = \kappa_{\mathrm{hi}}\,\alpha_{\mathrm{hi}}\,C_{\mathrm{hi}}\,V^2_{\mathrm{hi}}, \qquad r^{\mathrm{dec}}_{\mathrm{lo}} = \kappa_{\mathrm{lo}}\,\alpha_{\mathrm{lo}}\,C_{\mathrm{lo}}\,V^2_{\mathrm{lo}}, \tag{24}$$

writing C for C_{eff} to keep the subscripts readable. Dividing, the band's ratio is the product of per-factor widths

$$\frac{r^{\mathrm{dec}}_{\mathrm{hi}}}{r^{\mathrm{dec}}_{\mathrm{lo}}} \;=\; w_\kappa\,w_\alpha\,w_C\,w_V, \qquad w_x \equiv \frac{x_{\mathrm{hi}}}{x_{\mathrm{lo}}}, \qquad w_V \equiv \frac{V^2_{\mathrm{hi}}}{V^2_{\mathrm{lo}}}, \tag{25}$$

each $w_x \ge 1$. Taking every factor at its widest together makes this an upper bound, for the same reason the caveat

closing Section 3.3 gave for β itself; Appendix B treats both. Writing $W \equiv w_\kappa w_\alpha w_C w_V$ we can express the slope as

$$s \;=\; \frac{r_{\mathrm{hi}}^{\mathrm{dec}} - r_{\mathrm{lo}}^{\mathrm{dec}}}{r_{\mathrm{lo}}^{\mathrm{cov}}} \;=\; \frac{r_{\mathrm{lo}}^{\mathrm{dec}}}{r_{\mathrm{lo}}^{\mathrm{cov}}}\,\big(W-1\big). \qquad (26)$$

Note we never evaluate β from the factorisation; every number we report comes from measured band edges directly.

The overhead term. The overhead P_0 admits no comparably clean factorisation. It is a heterogeneous mix of static leakage, cooling control loops, firmware idle states, and platform I/O, so no single relation fixes the band width ΔE_0 from first principles.

4.2. Measurement on an NVIDIA A100

In order to measure β we require estimates of the three entries of the triple $\big(r_{\mathrm{hi}}^{\mathrm{dec}},\ r_{\mathrm{lo}}^{\mathrm{dec}},\ r_{\mathrm{lo}}^{\mathrm{cov}}\big)$, Equation (14), and the overhead width ΔE_0. Each experiment below exists to pin one of them. Experiment 1 supplies both declared edges $(r_{\mathrm{hi}}^{\mathrm{dec}}, r_{\mathrm{lo}}^{\mathrm{dec}})$, Experiment 2 supplies ΔE_0, and Experiment 3 supplies the covert floor $r_{\mathrm{lo}}^{\mathrm{cov}}$. Results are summarised in Table 1. Two further experiments are used later: Experiment 4 measures int8 transformer blocks (Section 5) and Experiment 5 sweeps the operand values at fixed precision (Appendix D).

Setup and provenance. The experiments below ran sequentially on one NVIDIA A100-SXM4-80GB in a single Slurm allocation. The platform denied clock locking and power capping, so DVFS is a recorded axis rather than a controlled one: each window logs the streaming-multiprocessor (SM) clock selected by the driver's frequency governor, and the bands are stratified into clock strata. That also puts V^2 beyond measurement here, so of the four factors of Equation (23) we measure three and carry $w_V \approx 1.5$ from device physics throughout; this is discussed as a limitation in Section 6.[2] Energy is read from the on-die NVML total-energy counter; each measured cell fills a $\sim$1.5 s window with back-to-back kernels on pre-allocated buffers and keeps the per-cell median over repeated windows. This is a non-adversarial, device-level calibration: the telemetry is trusted as a laboratory instrument. One GPU suffices to estimate the bands and the order of magnitude of β; extending the calibration to off-chip meters and multi-device systems is natural future work, taken up in Section 6. Appendix C gives the full protocol, the meter tolerance, and the provenance manifest, including which card measured which band.

Experiment 1—the declared band $(r_{\mathrm{hi}}^{\mathrm{dec}}, r_{\mathrm{lo}}^{\mathrm{dec}})$, and three of the four factor widths $(w_\kappa, w_\alpha, w_C)$. A compute-bound matmul (operation count $C = 2MNK$ known analytically[3]), swept factorially over precision (κ, fp32 down to int8), operand locality (C_{eff}, operands resident in vs. streamed through the L2 at a byte-identical shape), and operand distribution (α, zeros through adversarial-toggle).

quantity	what it is	source	value
(a) the factor widths of Equation (25)			
w_κ	precision	Exp. 1, cond.	13.5×
w_α	operands	Exp. 1, cond.	1.88×
w_C	locality	Exp. 1, cond.	1.04×
w_V	clock (DVFS)	*not swept; assumed*	≈1.5×
$w_\kappa w_\alpha w_C$	product	—	26.5×
(b) the edges the bound reads, nothing pinned			
$r_{\mathrm{hi}}^{\mathrm{dec}}$	fp32, expensive operands	Exp. 1	19.5
$r_{\mathrm{lo}}^{\mathrm{dec}}$	int8, zeros	Exp. 1	0.786
$r_{\mathrm{lo}}^{\mathrm{cov}}$	generic covert op	Exp. 3	0.51
ΔP_0	overhead band	Exp. 2	41.2 W

Table 1. Summary of results from Experiments 1–3. Values in pJ/op unless marked otherwise. Block (a) is the physics: the four factors of Equation (23) and how wide each one runs. "Cond." means the width is measured with the other three factors held fixed, so the widths are separable rather than pooled. Their product reproduces the measured span to 7%, and $w_\alpha w_C = 1.96$ predicts the fp16 sub-band's measured 2.00 to 2%. w_V is the exception and is flagged as such: Exp. 1 does not sweep the clock (the governor held it inside a ±3% window), so its width is a physics estimate, not a measurement. Block (b) is what the bound reads: the triple Equation (14) and the overhead width. These are the values a bare power meter concedes, with no format pinned on either declared edge and the covert floor priced for generic covert work. $C_{\max}$ and γ are read from published peaks rather than measured here.

The bound's two declared entries come off that sweep directly. With no format pinned they are its extremes, the most expensive cell (fp32, adversarial-toggle operands) against the cheapest (int8, zeros; Figure 5),

$$r_{\mathrm{hi}}^{\mathrm{dec}} = 19.5\ \mathrm{pJ/op}, \qquad r_{\mathrm{lo}}^{\mathrm{dec}} = 0.786\ \mathrm{pJ/op}. \qquad (27)$$

The sweep is a factorial so the widths of Equation (25) are also obtained for free. The 60 cells are every combination of five precisions, two localities and six operand distributions. To measure one factor's width, hold the other two axes fixed and take the ratio of the most to the least expensive cell as that factor runs over all its levels; the width is the median of that ratio across every such group of cells. This gives

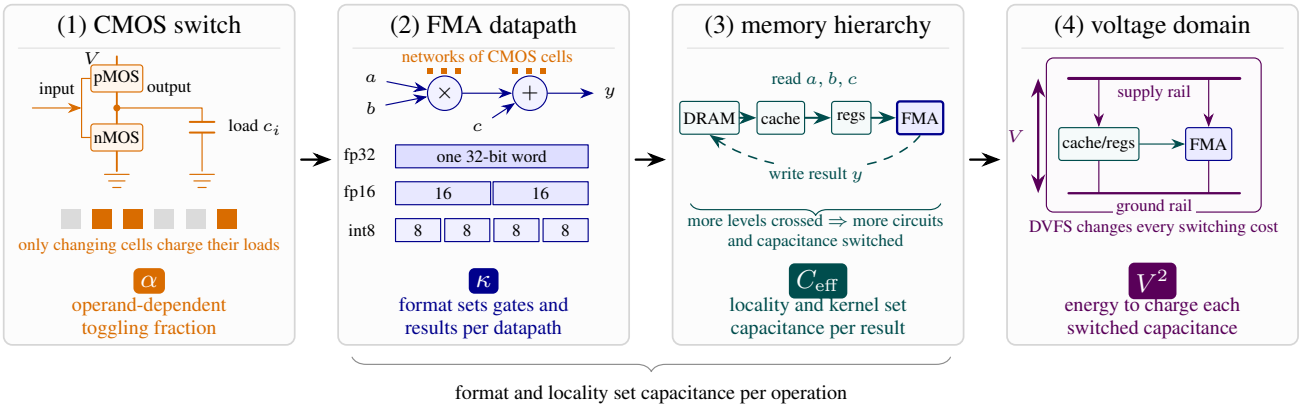
$$w_\kappa = 13.5, \qquad w_\alpha = 1.88, \qquad w_C = 1.04, \qquad (28)$$

with precision following the textbook ordering int8 < bf16 ≈ fp16 < tf32 < fp32. So precision dominates, operand values are worth a factor near two, and locality moves r by four per cent for this compute-bound kernel.[4] The clock is

[2] Supply voltages on modern accelerators sit near 1 V (Horowitz, 2014) and DVFS moves them by roughly ±10%; energy per operation scales as the ratio squared, $(1.1/0.9)^2 \approx 1.5$.

[3] The int8 cells retire integer multiply–adds, counted as nominal operations per the convention of Section 2.1.

[4] Per access, DRAM costs about a hundred times more than on-



$$r = \alpha \cdot \kappa \cdot C_{\text{eff}} \cdot V^2$$

f cancels from the marginal rate; leakage and cooling leave as the overhead P_0, not r

Figure 4. **The exchange rate, from transistors to the powered die.** Each physical scale contributes one factor to the per-operation energy of Equation (23). (1) A complementary pMOS–nMOS inverter charges or discharges its effective load c_i when its input changes; an operand determines what fraction α of all such cells toggle. (2) Networks of these cells implement the multiply and add of an FMA. The numeric format selects the word width (and supported packing), changing the logic engaged per arithmetic result and hence κ. (3) The FMA reads operands through the register–cache–DRAM hierarchy and writes its result back. Traversing more of that hierarchy switches additional array and peripheral circuits, buffers and interfaces, and interconnect; their combined dynamic cost is represented by C_{eff}. (4) Within a voltage domain, the supply places V across the arithmetic and interconnect loads, so DVFS contributes V^2 per unit capacitance. The clock f cancels from the marginal rate (Equation (39)) and the static draw—leakage, cooling—leaves as the overhead P_0. Multiplying the factors recovers Equation (23).

the one axis we could not hold fixed, so we check it rather than group on it. Comparing the median achieved clock across each axis's levels, the operand and locality levels ran at an identical clock and the precision levels to within 2.7%, so these widths are not clock artefacts. For the same reason the fourth width is missing: the governor held the clock inside 1335–1410 MHz across the whole sweep, a ±3% window, which is what keeps the three measured widths clean but leaves w_V no range to be measured over.

Multiplying the three measured widths gives $13.5 \times 1.88 \times 1.04 = 26.5$, against the 24.8× span the pooled cells actually show—so Equation (25) holds to 7%, the residual being factor interaction. Similarly, among the fp16 cells the precision never varies, so the spread should be the product of the two remaining widths, $w_\alpha w_C = 1.96$; those cells span [1.10, 2.19] pJ/op, a factor of 2.00. Appendix D performs a separate fp16-only sweep of the operand axis with a governor-selected clock and corroborates w_α independently, reaching 1.83 compared to the quoted value here of 1.88.

Experiment 2—the overhead band ΔP_0 (hence ΔE_0). The overhead is what the meter reads with no declared work running. We measure five variants chosen to bracket the plausible range: before any CUDA context, context up, context plus operand pool resident, immediately after a sustained heavy kernel, and after a 60 s cooldown. Each variant is measured over five repeated windows. The band is the lowest and highest of those 25 windows, not a range over per-variant medians. The lowest is a cold card before any context, at 62.9 W. The highest is a window taken immediately after the heavy kernel, at 96.2 W. That variant is a transient rather than a steady state: its five consecutive windows decay from 96.2 to 76.5 W as the card settles. We take its peak deliberately. An adversary can leave a device in that state, and the meter does not report which idle state it is reading, so the peak belongs in the envelope. Reducing each variant to a median instead would give $\Delta P_0 = 23.9$ W. The observed range [62.9, 96.2] W, stretched outward by the adopted ±5% meter tolerance[5], pins the overhead band to

$$P_0 \in [59.8,\ 101.0]\ \text{W}, \qquad \Delta P_0 = 41.2\ \text{W}. \tag{29}$$

chip memory (Horowitz, 2014); the per-operation effect is diluted by the kernel's bytes-per-operation, which is why w_C is small here. A memory-bound kernel would not enjoy that dilution, and w_C is the one width we would expect to move substantially with the workload.

[5]NVML publishes no accuracy specification for the total-energy counter `nvmlDeviceGetTotalEnergyConsumption` on GA100; the documented ±5% is for the power readings of the earlier Fermi and Kepler generations (NVIDIA Corporation, 2026). We adopt it as a working tolerance and apply it outward only, so it can only widen β (Appendix C). It accounts for 7.9 W of the width below; the remaining 33.3 W is genuine spread between idle states.

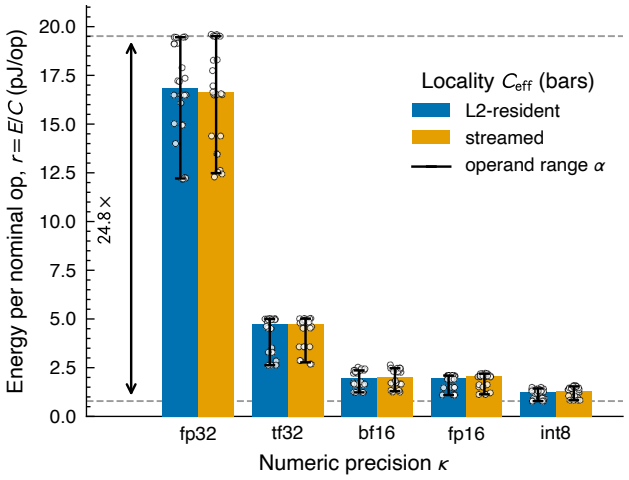


Figure 5. Experiment 1: Total energy quotient E/C for a fixed matmul kernel on an NVIDIA A100, swept factorially over precision κ (bar groups), operand locality C_{eff} (paired bars), and operand distribution α (whiskers); white points are the individual measured windows ($n = 240$ across 60 cells). Each bar is the median over the operand axis and its whisker that axis's full range, so the operand factor (a little under 2× inside every format, against locality's couple of per cent) is read off the figure rather than pooled into it. Dashed lines mark the widest setting's edges, both of them operand extremes: fp32 at its most expensive operands, int8 with zeros.

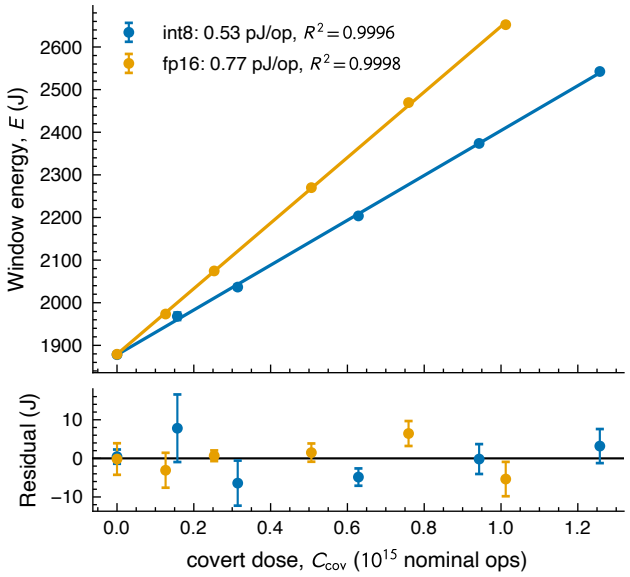


Figure 6. Experiment 3: window energy E vs. covert operation count C_{cov} for int8 and fp16 covert work, added to a fixed fp16 declared burst inside a fixed 10 s window. *Top:* per-cell medians with ±1 SD error bars over each cell's four untrimmed repeats (of five captured; the flagged high-tail repeat is excluded), against the fit $E = r_{\text{cov}}\, C_{\text{cov}} + E_0$; both slopes are linear to $R^2 > 0.999$ (0.53 pJ/op int8, 0.77 pJ/op fp16) and share an intercept matching the directly measured covert-free windows to 0.02%. *Bottom:* fit residuals.

Alongside the idle cells, Experiment 2 also sweeps a ladder of ten matmul shapes spanning a 60–90× range of sustained rate, as a consistency check on the affine forward map Equation (2): fitted like for like over the compute-bound shapes, the ladder's slope lands inside the fp16 exchange band of Experiment 1. The fitted slope is only ever a diagnostic, never an input to the bound (Appendix A).

Experiment 3—the marginal covert cost $r_{\text{lo}}^{\text{cov}}$. We construct a 10 s observation window. Inside it we run a fixed fp16 declared burst—a 2048^3 matmul on declared-typical operands, repeated enough times to keep the device busy for 30% of the window—then N_{cov} covert matmuls of a single format (int8 or fp16, each swept as its own arm), then an idle pad that fills out the remainder. The covert operands are zeros, the adversary's cheapest case. The two formats bracket what matters: int8 is the cheapest arithmetic the device offers, and fp16 matches the declared work, so covert operations are indistinguishable from declared ones by format alone. Only N_{cov} changes between windows: the declared burst, the shapes, the operand buffers, and the 10 s length are all held fixed.

To see how this experiment determines $r_{\text{lo}}^{\text{cov}}$, write the window energy out. Let e_{cov} be the energy of one covert matmul, t_{cov} its duration, c its nominal operation count, t_{dec} the declared burst's busy time, and P_{idle} the power drawn during the pad. Then

$$E = E_{\text{dec}} + N_{\text{cov}}\, e_{\text{cov}} + P_{\text{idle}}\big(T - t_{\text{dec}} - N_{\text{cov}} t_{\text{cov}}\big), \tag{30}$$

which collects into a constant plus a term linear in N_{cov},

$$E = \underbrace{E_{\text{dec}} + P_{\text{idle}}\big(T - t_{\text{dec}}\big)}_{\text{independent of } N_{\text{cov}}} + N_{\text{cov}} \underbrace{\big(e_{\text{cov}} - t_{\text{cov}} P_{\text{idle}}\big)}_{\text{the slope}}. \tag{31}$$

The first bracket does not depend on N_{cov}. The covert work amounts to $C_{\text{cov}} = N_{\text{cov}}\, c$ operations, so differentiating leaves only the second,

$$\frac{\partial E}{\partial C_{\text{cov}}} = \frac{e_{\text{cov}} - t_{\text{cov}} P_{\text{idle}}}{c} \equiv r^{\text{cov}}. \tag{32}$$

We sweep N_{cov} over six levels, from no covert work up to a covert busy fraction of 0.48, with five repeats per level in randomised order with drift-reference splices. Regressing window energy on covert nominal operations then gives the marginal cost directly (Figure 6): 0.53 pJ/op for int8 and 0.77 pJ/op for fp16, both linear to $R^2 > 0.999$. Pricing the same covert kernels the obvious way instead—total energy divided by total operations—gives 0.26 (int8) to 0.39 pJ/op (fp16) more. That excess is baseline power charged to the covert operations, which is what the fixed window differences away. Appendix A states precisely which estimand

each band edge requires, and why the covert edge must be this marginal cost rather than a quotient. We take the one-sided 95% lower confidence limits,

$$r_{\mathrm{lo}}^{\mathrm{cov}} = 0.51 \text{ pJ/op (int8)}, \qquad 0.76 \text{ pJ/op (fp16)}. \quad (33)$$

Because the covert cost divides the slack, under-pricing covert work is the security-safe direction.

One caveat is that, similar to the other experiments, clock locking is denied on this platform, so the governor is recorded rather than controlled here too. More covert work leaves a shorter idle pad, and the window-median clock rises with the load accordingly, from 1275 MHz with no covert work to 1410 MHz at the heaviest. However, what changes is the time spent at each clock frequency, not the operating point the kernels run at: every measured window, in both arms, records the same 1275 MHz minimum and 1410 MHz maximum. We would rather have locked the clock and compared every cell at one fixed setting. Two checks stand in for that. A clock inflating the cost of later operations would bend the response, and the fits show no systematic curvature; and the zero end of each fit lands on windows we measured directly with no covert work at all (Figure 6). We record the missing clock control as a limitation of the platform (Section 6).

Assembling β. We now combine the three experiments to evaluate the relaxed bound β (cf. Table 1):

- the declared band, from Experiment 1's fp16 cells: $[r_{\mathrm{lo}}^{\mathrm{dec}}, r_{\mathrm{hi}}^{\mathrm{dec}}] = [1.10,\ 2.19]$ pJ/op. This is narrower than the cross-format envelope of Table 1, which pins no format: we assume the protocols of Assumption 2.2 let the verifier confirm the declared work ran in its declared format, so the declared edges are the fp16 cells' own;
- the overhead width, from Experiment 2: $\Delta P_0 = 41.2$ W;
- the covert cost, from Experiment 3's marginal lower limits: $r_{\mathrm{lo}}^{\mathrm{cov}} = 0.51$ pJ/op for covert int8 ($\gamma = 2$) or 0.76 pJ/op for covert fp16 ($\gamma = 1$);
- capacity $C_{\max}$ and γ (Equation (19)), from the two formats' published peaks rather than the rates our kernels achieve;
- the declared utilisation h: we take the worst case $\beta^* = \max_h \beta(h)$ (Equation (22)).

Together these give, with covert int8,

$$\beta^* = 1.16 \quad \text{at} \quad h^* = 0.42, \quad (34)$$

and an energy-only width at full declared utilisation of $\beta_{\mathrm{en}}(1) = 2.39$. With covert work held to fp16 the pair is $\beta^* = 0.66$ at $h^* = 0.34$ and $\beta_{\mathrm{en}}(1) = 1.62$. Figure 7 traces $\beta(h)$ for both scenarios.

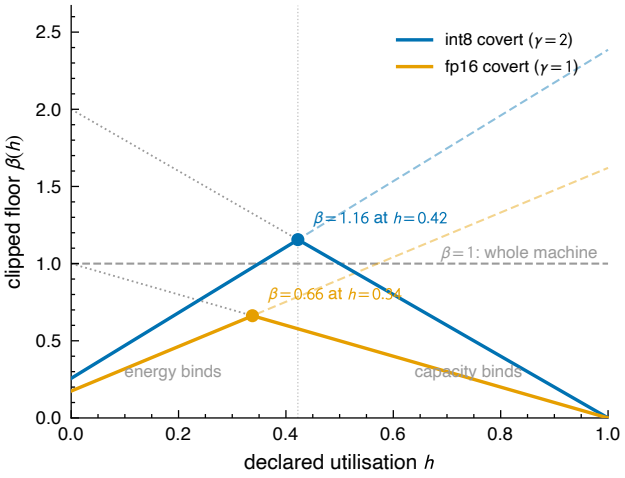


Figure 7. The relaxed bound $\beta(h)$ (Equation (21)) from the jointly feasible fp16-declared bands, for covert int8 ($\gamma = 2$, blue) and covert fp16 ($\gamma = 1$, orange), both priced at Experiment 3's marginal lower limits. Faint dashed lines are the unclipped energy-only widths $\beta_{\mathrm{en}}(h)$ ($\beta_{\mathrm{en}}(1) = 2.39$ and 1.62): energy is the binding constraint left of each crossing, capacity right of it, and both bind at h^*. Dotted lines are the capacity clips $(1 - h)\,\gamma$; markers are the worst cases of Equation (22).

Three remarks. First, $\beta_{\mathrm{en}}(1)$ is an energy-only diagnostic, not a physical floor: at full declared utilisation the capacity clip $(1 - h)\gamma$ is zero, so no covert work fits and $\beta(1) = 0$. Second, the clip is evaluated at published peak capacities rather than the rates our kernels achieve. Peaks give the covert stream more room than it can really use, which is the adversary-favourable direction: on our own measured peaks (246.9 Top/s fp16 and 307.7 Top/s int8, so $\gamma = 1.25$ rather than 2) the headline would read $\beta^* = 0.91$ rather than 1.16. Third, we can measure how much of this conceded width a real attack occupies. On a second card of the same SKU we ran pairs of runs matched in total energy: an honest run executing N declared fp16 iterations on expensive operands, and a cheating run executing the same iterations on cheap operands plus a covert int8 fill sized to draw the same energy. Every cheating run passes the no-alarm test of Equation (15) while executing 0.19–0.41 machines of covert work at $h = 0.10$–0.216. The witness therefore establishes $\beta_{\mathrm{ID}}(0.216) \geq 0.41$, against the relaxed model's $\beta(0.216) = 0.72$; the attack does not exhaust the bound because no single schedule reaches every edge of the band at once. Restricting the model to utilisations the attack can occupy ($h \lesssim 0.24$) would lower the worst case to $\approx$0.77, but that reflects our kernels rather than the adversary's best, so we do not quote it as a bound. Appendix B records the protocol.

5. The capability ladder

The headline of Section 4.2, $\beta^* = 1.16$ (Equation (34)), is a relaxed worst-case bound under Assumptions 2.1 and 2.2 alone: the verifier reads the power and checks the declared precision, and nothing else. In practice the verifier can make a series of reasonable checks and assumptions to remove ambiguity and push down β^*. Writing the energy-only width of Equation (18) with its inputs exposed,

$$\beta_{\text{en}}(h) \;=\; \beta_{\text{en}}\big(h;\, r_{\text{hi}}^{\text{dec}},\, r_{\text{lo}}^{\text{dec}},\, r_{\text{lo}}^{\text{cov}},\, \Delta E_0\big), \qquad (35)$$

we see that the triple of Equation (14) together with the overhead width are the only dials to turn to affect β_{en}.

The declared edges $r_{\text{hi}}^{\text{dec}}$ and $r_{\text{lo}}^{\text{dec}}$. These are accessible to the verifier, at least in principle, because they price work the prover has declared and the verifier may re-run. The factorisation of Equation (25) maps the available actions one can take to narrow that width: the declared precision is checked, so $w_\kappa = 1$, while the remaining factors $w_\alpha w_C w_V$ provide open degrees of freedom. Observations can remove the (w_V, w_C) freedom; re-executing the declared work on the verifier's own hardware measures $\bar{r}^{\text{dec}}$ directly, bypassing w_α rather than pinning it.

The covert price $r_{\text{lo}}^{\text{cov}}$. By definition, this is not accessible to the verifier; the prover still chooses the covert format, operands and locality freely, and the meter never resolves them. However, a threat model or an assumption about what the adversary is trying to do does introduce restrictions, for example restricting covert work to a measured workload class, or assuming that we know its numeric format.

The overhead width ΔE_0. Similarly, this dial is not accessible to the verifier but can be restricted through reasonable assumptions. For example, assuming that the card is executing throughout the window being priced, so its overhead cannot be that of a cold card.

The rest of this section takes these restrictions in turn. They compound. Each one keeps everything granted above it and turns one more dial, so the bound can only fall as we descend: from the headline $\beta^* = 1.16$, where the meter and the declared format are all the verifier has, to a conditional $\beta^* = 0.059$ at the bottom, with every restriction in place. Table 2 and Figure 8 walk down that ladder.

Observe the clock and locality (unpriced; dial $r_{\text{hi}}^{\text{dec}}$). This rung would set $w_V \to 1$ and $w_C \to 1$, leaving the declared band at its activity core w_α alone. Both are hardware-wide observables the power meter does not measure. We deliberately quote no number for it, for two reasons.

First, no verifier can do this today. Three routes might get there: the card could attest its own clock telemetry, the facility could lock its clocks by policy, or an external sensor could read the clock off the electromagnetic signature (Section 6). None of the three is currently deployable.

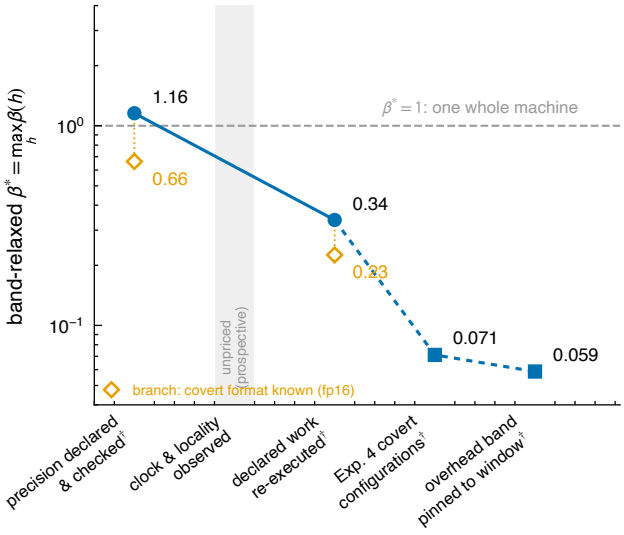


Figure 8. The capability ladder of Table 2 on a log scale; every plotted value is the relaxed worst case $\beta^* = \max_h \beta(h)$. Filled circles are capabilities the verifier gains; square rungs are forced by the threat model rather than granted by a channel; the shaded slot is the unpriced prospective clock/locality rung, and daggered labels carry the disclosed conditions of Table 2. Hollow diamonds are the covert-format branch, dropped from the two rungs it re-prices: it is not a continuation of the chain, and does not compose with the rungs below. The $\beta^* = 1$ line marks one whole machine of conceded ambiguity.

Second, and independently, we have not measured w_V. Neither Experiment 1 nor the operand sweep of Appendix D sweeps the clock. In both, the governor held it inside a $\pm 3\%$ window. So the ≈ 1.5 we carry for w_V is Section 4.2's device-physics estimate, not a measurement (Table 1).

Re-execute the declared work ($1.16 \to 0.34$, conditional; dial $r_{\text{hi}}^{\text{dec}}$). The verifier re-runs the declared work on its own hardware and measures what it costs. It reads the operands that actually ran, removing the prover's freedom to claim cheap data.

This collapses the declared band $r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}} \to \epsilon$, where ϵ is a small but non-zero residual that quantifies the difference between the verifier's hardware and the prover's hardware. The verifier's trusted cluster is not the monitored facility, and energy per operation depends on the accelerator, its firmware, and its cooling. So the measured rate transfers only up to this hardware gap.

We measure the easiest version of the gap by re-measuring every band edge on three further A100s of the same SKU. The edges agree to 2–3% across devices (Appendix C). Same-SKU re-execution therefore transfers bands at the few-per-cent level. However, across accelerator generations,

verifier gains	factor acted on, and the edges it leaves	β^*	$\beta_{\mathrm{en}}(1)$
precision declared & checked†	$w_\kappa = 1$; band $[1.10, 2.19]$	1.16	2.39
clock & locality observed	$w_V, w_C \to 1$ *(unpriced)*	—	—
declared work re-executed‡	*bypasses* w_α: band → gap	0.34	0.35
measured Experiment 4 transformer blocks‡	denominator: $0.51 \to 2.53$	0.071	0.072
overhead band pinned to the window‡	ΔP_0: $41.2 \to 31.3$ W	0.059	0.059
branch — covert format known to be fp16§			
at the precision rung	γ: $2 \to 1$; denominator $0.51 \to 0.76$	0.66	1.62
at the re-execution rung	*as above*, band → gap	0.23	0.24

Table 2. The capability ladder. Each row grants the verifier one more capability, cumulatively from the top, and moves one dial of Equation (35); the top row is the headline of Equation (34). Every row is the relaxed worst case $\beta^* = \max_h \beta(h)$ (Equation (22)) for the fp16-declared, int8-covert case study of Section 4.2, computed directly from the edges beside it; rate edges are in pJ/op. We include $\beta_{\mathrm{en}}(1)$, the energy-only width of Equation (18), as an additional diagnostic. The clock and locality row is left unpriced, for the two reasons given in Section 5. Disclosed conditions, inherited by every row below their first appearance: † — the declared work is executed in its declared format; ‡ — re-execution transfers at an observed operating point. § — the branch restricts covert work to fp16 instead of int8, and is quoted only at the two rungs where its denominator is measured: it acts on the same dial as the measured-workload row, so the two do not compose (Section 5).

firmware and cooling the gap is likely larger, and we have not measured it here.

To move forwards we assume that ϵ is a few per cent of the declared rate. We take $\epsilon \approx 0.05$ pJ/op, informed by the same-SKU spread but not measured across hardware, which gives $\beta^* = 0.34$ at $h^* = 0.83$. We treat this as a sensitivity analysis in ϵ rather than a measured bound: at the same-SKU spread itself (0.02–0.04 pJ/op) it reads $\beta^* = 0.29$–0.32; at our stated 0.05 pJ/op, $\beta^* = 0.34$; at a cross-vendor 0.2 pJ/op, $\beta^* \approx 0.54$.

Moreover, even in the case where $\epsilon = 0$, re-execution pays only at an observed operating point, cf. the unpriced clock rung above. The verifier measures the declared work at its own clock. If the prover's clock is unobserved the prover may run the declared work anywhere on the DVFS curve. The transfer error is then not the hardware gap but the whole DVFS factor, ~ 1.5. So a verifier with only the meter, declarations and re-execution, but no on-site sensing, stalls at $\beta^* \approx 1$. Re-executing the declared work buys it almost nothing that checking the precision had not already bought. Methods for pinning the clock are therefore a prerequisite for this rung, and no methods are currently deployable.

Restrict covert work to the measured transformer-block configurations ($0.34 \to 0.071$**; dial** $r_{\mathrm{lo}}^{\mathrm{cov}}$**; Experiment 4).** This is a workload-class restriction, granted by restrictions introduced by the threat model. Covert work is unseeable by definition: no declaration reaches it, so the unrestricted hardware edge remains the int8 marginal cost of 0.51 pJ/op. But the cheapest operations at that edge are operand zeros and in practice hidden zero-ops are no threat as they compute nothing. For this rung we instead restrict covert work to either of the two int8-quantised transformer-block configurations measured in Experiment 4, or to a class for which the verifier can independently justify the same marginal-cost floor. We do not claim that every useful computation costs at least this much, but we take it as a representative example of useful work beyond operand zeros.

We measured this by taking an int8 quantised transformer block built from one trained Pythia-6.9B layer, run on real text activations rather than synthetic patterns, with the quantisation scales applied. Every metered operation is counted, both the int8 matmuls and the fp16 attention. We swept that block as the covert dose inside Experiment 3's construction (Equation (31)), varying only the number of covert blocks between windows. We ran two shapes of the block, which differ in how much of the operation count is fp16 attention rather than int8 matmul, and so bound the per-operation cost from either side. Regressing window energy on covert nominal operations gives a marginal cost of 2.54 and 3.21 pJ/op for the two shapes, both fits linear to $R^2 > 0.999$. We carry the one-sided 95% lower limits, $r_{\mathrm{block}} = [2.53, 3.16]$ pJ/op.

For these two configurations, using the lower edge of the measured band raises the covert denominator from 0.51 to 2.53 pJ/op: every slack joule buys $\approx 5\times$ fewer operations than silicon-floor accounting would concede. The relaxed bound drops to $\beta^* = 0.071$ under this restricted scenario. Again we emphasise that this is a conditional case study, not a hardware-wide lower bound on the cost of useful computation.

Pin the overhead band to the window being priced ($0.071 \to 0.059$**; dial** ΔE_0**; Experiment 2).** Like the measured-workload rung, this rung is granted by restrictions introduced by the threat model. The band of Equation (29) spans Experiment 2's twenty-five idle windows. Its low edge, 62.9 W, is the cold-card cell—the device before any CUDA context exists. But in practice the card is not cold. The declared burst runs for $h\,T$ and covert work fills the rest of the window. The card is therefore executing from

one end of the window to the other, and its operands stay in memory throughout. A card in that state cannot draw a cold card's overhead. Restricting the band to the fifteen windows in which a context and operand pool were resident lifts the low edge to 73.4 W. The high edge does not move: the 96.2 W peak is the post-heavy-kernel transient, and the priced window's declared burst *is* a heavy kernel. With the same $\pm 5\%$ meter tolerance,

$$P_0 \in [69.7,\ 101.0]\ \mathrm{W}, \qquad \Delta P_0 = 31.3\ \mathrm{W}, \tag{36}$$

which is 0.76 of the published width, so the rung falls from 0.071 to 0.059. The restriction is not specific to the bottom of the ladder: at the re-execution rung it reads 0.28 in place of 0.34, and at the headline 1.12 in place of 1.16, where the declared band and not the overhead carries the width.

Even after pinning, most of the bottom rung is overhead. The overhead term $b = \Delta P_0/(r_{\mathrm{lo}}^{\mathrm{cov}}\, F_{\mathrm{max}}^{\mathrm{dec}})$ is about 70% of β^* at the bottom rung (0.040 of 0.059) and at the re-execution rung (0.195 of 0.28). Reporting P_0 directly, over a trusted on-site channel rather than bracketing it, would leave only the meter tolerance, $\Delta P_0 \approx 8.5$ W, and a bottom rung of $\beta^* = 0.030$.

Branch: know the covert format ($1.16 \to 0.66$; dials $r_{\mathrm{lo}}^{\mathrm{cov}}$ and γ). This restriction is enabled by the threat model. We assume that the covert numeric format is known. This is a reasonable assumption: if compute capacity is being stolen to train a frontier model, that training is highly likely to run in fp16 or bf16 with fp32 accumulation. int8 pre-training is not a viable route to the capability an adversary wants.

Restricting covert work to fp16 moves two things. First, the covert denominator rises from the int8 marginal edge to the fp16 one, $0.51 \to 0.76$ pJ/op, both measured in Experiment 3 (Section 4.2). That is a factor $\approx$1.5 on the declared and the overhead term alike, since both carry $r_{\mathrm{lo}}^{\mathrm{cov}}$ in their denominator. Second, the capacity ratio falls, $\gamma = 2 \to 1$: covert operations no longer retire at twice the declared rate, so the clip becomes $1 - h$. At the headline rung the two together give $\beta^* = 0.66$. Carried down to the re-execution rung the branch reads 0.23 in place of 0.34. Figure 7 traces both scenarios.

6. Discussion

We have shown how a power meter constrains hidden compute only to an identified set. For a meter that also checks the declared precision, matched schedules establish $\beta^*_{\mathrm{ID}} \geq 0.41$, while the relaxed bound gives $\beta^*_{\mathrm{ID}} \leq \beta^* = 1.16$. The capability ladder can lower the relaxed bound to 0.059, but only under all of its stated conditions: the declared operating point and locality are observed, re-execution transfers with the assumed 0.05 pJ/op gap, covert work is restricted to Experiment 4's two measured transformer-block configurations, and the overhead is that of a resident card.

Limitations. The above claim on the value of β has a series of caveats as follows. (i) The empirical values of Sections 4 and 5 come from one vendor, one GPU model, one driver, and one cluster. Consequently the quoted values of β are A100-specific, though the framework and closed form of Section 3 are general. (ii) The covert denominator behind the headline, $r_{\mathrm{lo}}^{\mathrm{cov}} = 0.51$ pJ/op, is measured, but on synthetic operands: its cheapest cell is int8 on zeros, which compute nothing. Experiment 4 measures only two Pythia transformer-block configurations on real text activations. The bottom rungs rest on the lower edge of that measurement; it is not a universal lower bound for useful computation. (iii) The hardware gap behind the re-execution rung ($\approx$0.05 pJ/op) is an assumption. The same-SKU piece is grounded: band edges agree to 2–3% across three A100s (Appendix C). However, the cross-hardware gap (different accelerators, firmware, cooling) remains unmeasured. (iv) The platform denied clock locking and so DVFS was never swept. Accordingly the voltage factor $w_V \approx 1.5$ of Equation (25) rides on literature numbers rather than our own. The declared band was therefore sampled with the governor holding the clock inside a $\pm 3\%$ window, whereas the threat model gives the prover the clock: a prover free to slide DVFS reaches a wider declared band and possibly a cheaper covert floor, raising both s and b in Equation (18). (v) Energy was read from on-die telemetry, whereas in practice the verifier reads a retrofitted meter on the power feed. The on-die counter carries no published GA100 accuracy specification, so the $\pm 5\%$ tolerance we apply is adopted from NVML's documented Fermi/Kepler figure, and the $\sim$100 ms power-telemetry cadence is an empirical characterisation (Yang et al., 2024), not a vendor specification. That retrofitted meter carries its own calibration band, and it also sees conversion and cooling draw the die counter never reports. Both widen ΔP_0, and so widen β. (vi) We measured one device, whereas a facility is many devices behind shared power delivery and cooling. We do not offer a facility-level, multi-device version of Equation (18). Each device carries its own capacity clip and its own utilisation. The prover chooses how the declared work is spread across devices and formats, so the width of the identified set depends on an allocation the prover is free to pick. How to make the multi-device bound precise is an open question. (vii) The relaxed bound is a worst case in a strong sense. It prices a window in which the declared work is at its cheapest, the overhead at its lowest, and the covert work at its cheapest—all three at the same time. We do not show that any single schedule can do all three at once. If none can, β_{ID} is narrower than the β we quote (Equation (43)), and we have conceded the adversary more than the silicon allows.

Richer telemetry. Throughout this work we have dealt exclusively with time-integrated power, i.e. energy. Because $r(t)$ and $P_0(t)$ may vary arbitrarily within their bands, the time-resolved power trace carries no information about total compute beyond the integral. However, real hardware is not perfectly free and does not vary arbitrarily. Instead power-ramp limits, DVFS governor dynamics, the instantaneous clip $F(t) \leq F_{\max}$, and any declared schedule the prover commits to all constrain how the trace can move. A verifier who models these constraints can likely extract more from the resolved trace than from its integral. The time-resolved trace also carries different information such as workload cadence or training-step periodicity. This could be useful for attacking different questions about what kind of compute ran (e.g. training vs. inference) rather than how much; that question needs different machinery and we do not treat it here.

Moreover, we have focused exclusively on a single analogue measurement channel: power. A verifier with richer instrumentation or multiple simultaneous channels (e.g. per-rail power, electromagnetic, thermal), cryptographically authenticated telemetry (the protocols of Assumption 2.2), or independently controlled measurements the prover cannot shape (locked clocks, tamper-resistant attested counters) would likely be able to further constrain the total covert compute. For example, take the clock. The power meter cannot see it, which is why Section 5 leaves the clock-and-locality rung unpriced. An antenna beside the rack might see it instead. A circuit switching at frequency f radiates at f and its harmonics, and magnetic emanations from a GPU's power delivery are measurable in practice (Maia et al., 2022). A verifier who could read the clock this way would get the voltage too, since that follows from the hardware's V–f table. The DVFS factor w_V would fall to 1, and the declared band would narrow to its activity core w_α alone. We defer the study of multiple measurement channels to future work.

7. Conclusion

So, can power draw constrain covert compute? Only weakly on its own. Against a meter that also knows the declared numeric format, matched schedules demonstrate at least 0.41 machines of physical ambiguity, while the relaxed bound caps the worst case at 1.16: $0.41 \leq \beta^*_{\mathrm{ID}} \leq 1.16$.

However, the meter is rarely all a verifier has. Additional information and threat-model restrictions can narrow the relaxed bound. If the declared operating point and locality are observed, re-execution transfers with the stated hardware gap, covert work belongs to one of the two measured transformer-block configurations (or a separately justified class with the same cost floor), and the card has resident-workload rather than cold-card overhead, then these restrictions bring β^* to 0.059. This is a case study of what those restrictions buy, not a universal bound on arbitrary useful computation.

The lowering of β^* brought by additional restrictions strongly motivates the case for reading power alongside other channels rather than by itself. The clock is the clearest example; the meter cannot see it, but an antenna beside the rack might. A verifier who can read the clock fixes the voltage too and narrows the declared band to its activity core. Pricing that rung, replacing the on-die counter with a retrofitted meter on the power feed, and carrying the bound from one device to a whole facility are natural next steps.

Declaration on AI usage

Following the disclosure principles of the Leiden Declaration on Artificial Intelligence and Mathematics (Alper et al., 2026), we record the AI tools this work relied on. Large language models (primarily Claude Opus 5, with additional inputs from Claude Fable 5 and GPT-5.6-Sol) were used for writing, refactoring and debugging the benchmark and analysis code. The mathematics was derived without LLM assistance; LLMs were used to check the consistency of the algebra. Every reported number is regenerated from the committed records by scripts in the repository, and the authors take full responsibility for the content of this paper.

References

Alper, J., Barany, M., Chavarri Villarello, A., Dahmen, S., Dean, W., Ganapathy, K., Harris, M., Holmes, D., Jamnik, M., Kelk, S., Kra, B., Martin, U., Naskręcki, B., Ochigame, R., Portegies, J., and Schmitt, J. Leiden Declaration on Artificial Intelligence and Mathematics, June 2026. URL https://doi.org/10.5281/zenodo.20302944.

Ansari, S. Hardware-level governance of AI compute: A feasibility taxonomy for regulatory compliance and treaty verification, April 2026. URL https://arxiv.org/abs/2604.04712.

Arefin, S. E. and Serwadda, A. Exploiting HDMI and USB ports for GPU side-channel insights. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (CODASPY)*, pp. 131–142, 2025. doi: 10.1145/3714393.3726511.

Baker, M., Kulp, G., Marks, O., Brundage, M., and Heim, L. Verifying international agreements on AI: Six layers of verification for rules on large-scale AI development and deployment, July 2025. URL https://arxiv.org/abs/2507.15916.

Bengio, Y., Clare, S., Prunkl, C., Andriushchenko, M., Bucknall, B., et al. International AI Safety Report 2026, Febru-

ary 2026. URL https://arxiv.org/abs/2602.21012.

Chandrakasan, A. P., Sheng, S., and Brodersen, R. W. Low-power CMOS digital design. *IEEE Journal of Solid-State Circuits*, 27(4):473–484, 1992. doi: 10.1109/4.126534.

Chatterjee, N., O'Connor, M., Lee, D., Johnson, D. R., Keckler, S. W., Rhu, M., and Dally, W. J. Architecting an energy-efficient DRAM system for GPUs. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 73–84, 2017. doi: 10.1109/HPCA.2017.58.

Chaudhuri, A., Shukla, S., Bhattacharya, S., and Mukhopadhyay, D. "Energon": Unveiling transformers from GPU power and thermal side-channels. In *2025 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 1–9, 2025. doi: 10.1109/ICCAD66269.2025.11240793.

Choukse, E., Warrier, B., Heath, S., Belmont, L., Zhao, A., Khan, H. A., Harry, B., Kappel, M., Hewett, R. J., Datta, K., et al. Power stabilization for AI training datacenters, August 2025. URL https://arxiv.org/abs/2508.14318.

Clark, S. S., Ransford, B., Rahmati, A., Guineau, S., Sorber, J., Xu, W., and Fu, K. WattsUpDoc: Power side channels to nonintrusively discover untargeted malware on embedded medical devices. In *2013 USENIX Workshop on Health Information Technologies (HealthTech 13)*. USENIX Association, 2013. URL https://www.usenix.org/conference/healthtech13/workshop-program/presentation/clark.

Gangwal, A., Piazzetta, S. G., Lain, G., and Conti, M. Detecting covert cryptomining using HPC. In *Cryptology and Network Security (CANS 2020)*, Lecture Notes in Computer Science, pp. 344–364. Springer, 2020. doi: 10.1007/978-3-030-65411-5_17.

Gao, Y., Qiu, H., Zhang, Z., Wang, B., Ma, H., Abuadbba, A., Xue, M., Fu, A., and Nepal, S. DeepTheft: Stealing DNN model architectures through power side channel. In *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 3311–3326, 2024. doi: 10.1109/SP54263.2024.00250.

Garg, S., Goel, A., Jha, S., Mahloujifar, S., Mahmoody, M., Policharla, G.-V., and Wang, M. Experimenting with zero-knowledge proofs of training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1880–1894, 2023. doi: 10.1145/3576915.3623202.

Gong, C., Liu, B., and Li, M. Hollow-LLM attack: Computationally trivial weights in zero-knowledge verification of LLM inference, 2026.

Heim, L. and Koessler, L. Training compute thresholds: Features and functions in AI regulation, 2024. URL https://arxiv.org/abs/2405.10799.

Horowitz, M. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*, pp. 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323.

Hu, X., Liang, L., Li, S., Deng, L., Zuo, P., Ji, Y., Xie, X., Ding, Y., Liu, C., Sherwood, T., and Xie, Y. DeepSniffer: A DNN model extraction framework based on learning architectural hints. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 385–399. ACM, 2020. doi: 10.1145/3373376.3378460.

Jia, H., Yaghini, M., Choquette-Choo, C. A., Dullerud, N., Thudi, A., Chandrasekaran, V., and Papernot, N. Proof-of-learning: Definitions and practice. In *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 1039–1056, 2021. doi: 10.1109/SP40001.2021.00106.

Komargodski, I. and Weinstein, O. Proofs of useful work from arbitrary matrix multiplication. Cryptology ePrint Archive, Paper 2025/685, 2025. URL https://eprint.iacr.org/2025/685.

Latif, I., Newkirk, A. C., Carbone, M. R., Munir, A., Lin, Y., Koomey, J., Yu, X., and Dong, Z. Single-node power demand during AI training: Measurements on an 8-GPU NVIDIA H100 system. *IEEE Access*, 13:61740–61747, 2025. doi: 10.1109/ACCESS.2025.3554728. URL https://www.osti.gov/biblio/2555926.

Maia, H. T., Xiao, C., Li, D., Grinspun, E., and Zheng, C. Can one hear the shape of a neural network? Snooping the GPU via magnetic side channel. In *31st USENIX Security Symposium (USENIX Security 22)*, pp. 4383–4400. USENIX Association, 2022. URL https://www.usenix.org/conference/usenixsecurity22/presentation/maia.

Monfared, S. K., Ganji, F., Holcomb, D., and Tajik, S. Timing and memory telemetry on GPUs for AI governance, 2026. URL https://arxiv.org/abs/2602.09369.

Naghibijouybari, H., Neupane, A., Qian, Z., and Abu-Ghazaleh, N. Rendered insecure: GPU side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2139–2153. ACM, 2018. doi: 10.1145/3243734.3243831.

NVIDIA Corporation. NVIDIA A100 tensor core GPU architecture. Whitepaper, 2020. URL

https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf. Dense tensor-core peaks: 19.5 TFLOP/s fp32, 156 TFLOP/s tf32, 312 TFLOP/s fp16/bf16, 624 TOPS int8.

NVIDIA Corporation. NVIDIA management library (NVML) API reference. Online documentation, 2026. URL https://docs.nvidia.com/deploy/nvml-api/. Device queries `nvmlDeviceGetPowerUsage` and `nvmlDeviceGetTotalEnergyConsumption`. The stated +/- 5% accuracy is given for Fermi and Kepler power readings; no accuracy specification is given for the total-energy counter on GA100. Accessed 2026-10-05.

O'Gara, A., Kulp, G., Hodgkins, W., Petrie, J., Immler, V., Aysu, A., Basu, K., Bhasin, S., Picek, S., and Srivastava, A. Hardware-enabled mechanisms for verifying responsible AI development. In *ICML 2025 Workshop on Technical AI Governance (TAIG)*, 2025. URL https://arxiv.org/abs/2505.03742.

Pallipadi, V. and Starikovskiy, A. The ondemand governor: Past, present, and future. In *Proceedings of the Linux Symposium*, volume 2, pp. 223–238, 2006. URL https://www.kernel.org/doc/ols/2006/ols2006v2-pages-223-238.pdf.

Patel, P., Choukse, E., Zhang, C., Goiri, Í., Warrier, B., Mahalingam, N., and Bianchini, R. POLCA: Power oversubscription in LLM cloud providers, 2023. URL https://arxiv.org/abs/2308.12908.

Rahman, R. and Tajdari, S. Detecting hidden ML training with zero-overhead telemetry, 2026. URL https://arxiv.org/abs/2606.19262.

Reuel, A., Bucknall, B., Casper, S., Fist, T., Soder, L., Aarne, O., Hammond, L., Ibrahim, L., Chan, A., Wills, P., Anderljung, M., Garfinkel, B., Heim, L., Trask, A., Mukobi, G., Schaeffer, R., Baker, M., Hooker, S., Solaiman, I., Luccioni, A. S., Rajkumar, N., Moës, N., Ladish, J., Bau, D., Bricman, P., Guha, N., Newman, J., Bengio, Y., South, T., Pentland, A., Koyejo, S., Kochenderfer, M. J., and Trager, R. Open problems in technical AI governance. *Transactions on Machine Learning Research*, 2025. URL https://arxiv.org/abs/2407.14981.

Sastry, G., Heim, L., Belfield, H., Anderljung, M., Brundage, M., Hazell, J., O'Keefe, C., Hadfield, G. K., Ngo, R., Pilz, K., Gor, G., Bluemke, E., Shoker, S., Egan, J., Trager, R. F., Avin, S., Weller, A., Bengio, Y., and Coyle, D. Computing power and the governance of artificial intelligence, February 2024. URL https://arxiv.org/abs/2402.08797.

Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., and Villalobos, P. Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2022. doi: 10.1109/IJCNN55064.2022.9891914.

Shavit, Y. What does it take to catch a Chinchilla? Verifying rules on large-scale neural network training via compute monitoring, 2023. URL https://arxiv.org/abs/2303.11341.

Sun, H., Li, J., and Zhang, H. zkLLM: Zero knowledge proofs for large language models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 4405–4419, 2024. doi: 10.1145/3658644.3670334.

Tang, B. J., Chen, Q., Weiss, M. L., Frey, N., McDonald, J., et al. The MIT Supercloud workload classification challenge. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 708–714, 2022. doi: 10.1109/IPDPSW55747.2022.00122.

The White House. Executive Order 14110: Safe, secure, and trustworthy development and use of artificial intelligence. Federal Register 88 FR 75191, 2023. URL https://www.federalregister.gov/documents/2023/11/01/2023-24283/.

The White House. Executive Order 14148: Initial rescissions of harmful executive orders and actions. Federal Register 90 FR 8237, Doc. 2025-01901, 2025. URL https://www.federalregister.gov/documents/2025/01/28/2025-01901/.

Wei, J., Zhang, Y., Zhou, Z., Li, Z., and Al Faruque, M. A. Leaky DNN: Stealing deep-learning model secret with GPU context-switching side-channel. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 125–137. IEEE, 2020. doi: 10.1109/DSN48063.2020.00031.

Xu, H., Gong, C., Liu, B., Zheng, H., Chen, B., and Li, M. Wave: Leveraging architecture observation for privacy-preserving model oversight. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 2212–2231, 2026. doi: 10.1145/3779212.3790247.

Yang, Z., Adámek, K., and Armour, W. Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU's built-in power sensor. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–17, 2024. doi: 10.1109/SC41406.2024.00028.

A. From the CMOS power law to the exchange rate

This appendix derives the two models the body text adopts: the affine forward map Equation (2) sketched in Section 2.1, and the factored exchange rate Equation (23) of Section 4.1. It closes by collecting the measurements that support the affine map, and by showing that the security bound needs less than it—a weaker, one-sided condition—and what that condition demands of how each band edge is measured.

Start from the power law. Expanding the static term of Equation (1), we split total chip power into a dynamic (switching) part and a static part,

$$P = P_{\mathrm{dyn}} + P_{\mathrm{stat}}, \qquad P_{\mathrm{dyn}} = \alpha\, C\, V^2 f, \qquad P_{\mathrm{stat}} = V I_{\mathrm{leak}} + P_{\mathrm{fixed}}, \tag{37}$$

with α the activity factor, C the total capacitance of the logic (so αC is the capacitance switched per cycle), V the supply voltage, f the clock, $V I_{\mathrm{leak}}$ the leakage power, and P_{fixed} the cooling, I/O, and idle draw. The forward model Equation (2), $P = rF + P_0$, is a straight line in the operation rate F, so the exchange rate is its slope, i.e. the marginal energy per operation, and the overhead is the intercept,

$$r = \frac{\partial P}{\partial F}, \qquad P_0 = P\big|_{F=0}. \tag{38}$$

Note that one cannot simply plot measured power against measured rate and read r off the slope. The card does not have one r: the cost per operation depends on what is executing (e.g. the format, the kernel, where the operands live), and the choices that move a workload's achieved rate F also move its cost per operation r. Points collected across workloads therefore lie on different lines through a shared intercept, and a slope fitted through them averages over workloads rather than pricing any one of them (Experiment 2's rate ladder shows exactly this: when the fit is opened beyond the compute-bound shapes, its scatter grows sharply while its slope barely moves). At a fixed workload, power *is* linear in the rate and its slope is r (cf. Equation (39) below), but each workload carries its own value of r.

The dynamic term gives r. The operation rate is $F = N_{\mathrm{op}}\, f$, where N_{op} is the number of nominal operations the datapath retires per cycle at the operating format. At a fixed format, $\partial f/\partial F = 1/N_{\mathrm{op}}$, and the exchange rate of Equation (38) follows by the chain rule, holding α, C, and V fixed,

$$r \;=\; \frac{\partial P_{\mathrm{dyn}}}{\partial F} \;=\; \frac{\partial P_{\mathrm{dyn}}}{\partial f}\,\frac{\partial f}{\partial F} \;=\; \frac{\alpha\, C\, V^2}{N_{\mathrm{op}}} \;=\; \alpha \left(\tfrac{C}{N_{\mathrm{op}}}\right) V^2. \tag{39}$$

Note that the clock cancels: r depends on what each switching event costs and how many operations it delivers, not on how fast the operations run. The slope r is constant in F, so $P_{\mathrm{dyn}} = rF$ is linear through the origin, exactly the affine form the forward model Equation (2) assumes and Experiment 2's rate ladder checks (Section 4.2). The capacitance switched per operation, C/N_{op}, has two structural sources: the arithmetic datapath, whose width is set by the numeric format, and the data-movement path, set by where the operands reside and which kernel runs. Factoring it accordingly,

$$\frac{C}{N_{\mathrm{op}}} \;=\; \kappa(\text{precision})\; C_{\mathrm{eff}}(\text{locality, kernel}), \tag{40}$$

with κ collecting the format scale (datapath width and the operation packing N_{op}) and C_{eff} the locality and kernel structure. Substituting Equation (40) into Equation (39) gives the model Equation (23). We emphasise this is not a unique factorisation. Instead it is a regrouping of the switching contributions by physical origin; κ, α, and C_{eff} are all switching quantities, grouped by cause so that each factor corresponds to one property of the workload that a verifier could separately check or constrain: the numeric format, the operand data, and the memory locality and kernel.

Short-circuit sits in r, inside the switched capacitance. Chandrakasan et al. (1992) also carry a *short-circuit* term for the direct-path current that flows while an input transitions and the NMOS and PMOS networks briefly conduct together. It is triggered by the same switching events as P_{dyn}, so it scales with the operation rate and survives the marginal derivative of Equation (38). In a well-designed circuit it is a small fraction of the switching power, so we fold it into the effective switched capacitance C_{eff} of Equation (40): it lands in r, but adds nothing new for a verifier to check.

Leakage sits in P_0, not r. The static power P_{stat} of Equation (37) flows whether or not the machine computes, so $\partial P_{\text{stat}}/\partial F = 0$: by Equation (38) it contributes nothing to the exchange rate r and is entirely the intercept P_0, where the derivation of Equation (10) already places it and carries it as a bounded nuisance band. A leakage-per-operation term $r_{\text{leak}} = P_{\text{stat}}/F$ would appear only if r were defined as the average energy per operation, P/F, which mixes in the intercept and is operating-point dependent; we keep the marginal definition Equation (38), consistent with Equation (2), and so carry no separate leakage term in r.

Is the affine map true? The derivation above gives the model theoretically; the measurements also support it directly, in three places. Experiment 2's rate ladder is the across-workload check: restricted to the compute-bound shapes, where like is compared with like, metered power against sustained operation rate is linear at $R^2 = 0.99$, and the fitted slope lands inside the fp16 exchange band $[1.10,\ 2.19]$ pJ/op that Experiment 1 measures independently by total quotient (Section 4.2). Experiments 3 and 4 test the covert term at fixed declared work: window energy is linear in the covert dose to $R^2 > 0.999$ (Figure 6)—affine in the one argument whose marginal cost the security bound uses. And the idle cells of Experiment 2 pin the intercept directly (Equation (29)). What no measurement certifies is one affine map across all workloads at once: as noted above, each workload carries its own slope, and the pooled ladder fit degrades accordingly. So Equation (2) holds workload by workload: within one workload power is a straight line in the rate, and different workloads have different slopes. As the next paragraph shows, that is all the bound needs.

The bound needs less than the affine map. Why is the workload-by-workload form enough? Because the security bound does not, in fact, require one affine map across all workloads; it needs one weaker, one-sided condition. To state it, write the true draw as $P = g_x(F_{\text{dec}}, F_{\text{cov}})$, some device-dependent function of what the silicon executes, with the subscript x standing for the execution details the verifier never sees—no linearity assumed. The condition is then

$$g_x(F_{\text{dec}}, F_{\text{cov}}) - g_x(F_{\text{dec}}, 0) \ \geq\ r_{\text{lo}}^{\text{cov}}\, F_{\text{cov}}, \tag{41}$$

for every admissible execution g_x: adding covert work at rate F_{cov} on top of the declared work costs at least $r_{\text{lo}}^{\text{cov}}$ joules per covert operation, however the silicon runs it. The derivation of Equation (17) converts the honest energy slack into covert operations by dividing by $r_{\text{lo}}^{\text{cov}}$—so under Equation (41) alone the bound holds, with no global affine map anywhere. The affine model is the special case $g_x = P_0 + r^{\text{dec}}F_{\text{dec}} + r^{\text{cov}}F_{\text{cov}}$, which additionally makes $\beta_{\text{en}}(h)$ linear in h.

The condition Equation (41) dictates how each band edge must be measured. The declared edges enter Equation (17) only through their difference $r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}$ at a common saturated rate, so any baseline amortised equally into both edges cancels and a measured total quotient E/C is the correct estimand for the numerator (the overhead enters once, separately, as ΔE_0). The covert edge is different: it must be a genuine one-sided lower bound on the incremental cost in Equation (41), and a total quotient cannot be one, because a quotient amortises baseline power (e.g. leakage, cooling, idle draw) that an added covert operation does not pay. Every covert edge the bound uses is therefore identified by a dose–response intervention (Experiment 3 for the generic int8 edge, Experiment 4 for the useful-operand edge), never by a quotient. The same logic is why Table 1 keeps $r_{\text{lo}}^{\text{dec}}$ and $r_{\text{lo}}^{\text{cov}}$ distinct even though they describe the same physical case, int8 on zeros. The marginal figure is the cheaper of the two, and a cheaper covert denominator concedes the adversary more operations per slack joule, so carrying it widens the bound—the safe direction.

B. The matched-energy witness protocol

This appendix establishes two results that bracket the physical ambiguity. The model bound $\beta^* = 1.16$ is an *upper* limit: it puts each band at its worst edge independently, and no single schedule is guaranteed to reach all of those edges at once. A constructed cheating schedule, accepted by the meter, puts a measured *lower* limit of 0.41 machines on the same ambiguity. The physical identified set against a format-checked meter therefore obeys

$$0.41 \ \leq\ \beta^*_{\text{ID}} \ \leq\ \beta^* = 1.16, \tag{42}$$

and this appendix records both halves of that bracket.

The upper end: β treats the bands independently. The maximisation that produced Equation (21) lets each of the three bands range freely, so the prover may sit at $r_{\text{lo}}^{\text{dec}}$, $E_{0,\text{lo}}$ and $r_{\text{lo}}^{\text{cov}}$ all at once. Nothing in the derivation guarantees a single schedule reaches all three edges at once. For equality we must assume that there exist admissible workloads

attaining $(\bar{r}^{\mathrm{dec}}, E_0, \bar{r}^{\mathrm{cov}}) = (r_{\mathrm{lo}}^{\mathrm{dec}}, E_{0,\mathrm{lo}}, r_{\mathrm{lo}}^{\mathrm{cov}})$ jointly and under shared constraints: the *same* observation window T, the *same* declared work C_{dec}, and one device in a *common thermal and clock state* whose time the two streams share at their *achieved* rates. Treating the bands independently is a relaxation of what a device can schedule, and

$$\beta_{\mathrm{ID}}(h) \;\le\; \beta(h), \tag{43}$$

with equality only if one schedule can reach all three edges at once. Conceding the adversary more than the silicon may allow is the safe direction for a security claim, but it leaves the gap in Equation (43) unquantified. (One bookkeeping rule keeps the bands honest: every number entering the bound comes from one scenario a device could actually run—one declared format and one covert format sharing one window. We never mix formats, for example by taking the declared cost from one format's most expensive cells and the covert capacity from another.)

The lower end: a schedule the meter accepts. Closing the gap from below takes a constructed schedule. The matched-energy witness of Section 4.2 holds the trusted declaration and the observation window fixed: run A executes N declared fp16 iterations on expensive operands (the $r_{\mathrm{hi}}^{\mathrm{dec}}$ edge) and idles to a fixed $T = 10\,\mathrm{s}$ window; run B executes the *same* N iterations on cheap operands plus a covert int8 fill tuned to match A's energy. The acceptance rule is one-sided: run B is accepted if its window energy stays below run A's honest mean plus three standard deviations of A's repeat scatter. At $h = 0.10$, 0.15 and 0.216 the accepted covert load reaches 0.19, 0.29 and 0.41 machines of declared-format capacity: measured, capacity- and time-feasible lower bounds on the ambiguity. The match is tight: at $h = 0.10$ the arms agree to $|\Delta E|/E = 0.9\%$ over eight interleaved repeats (1291.3 ± 4.4 against $1279.3 \pm 5.6\,\mathrm{J}$), while B hides 3.5×10^4 covert int8 iterations behind the no-alarm rule. And the witness is conservative: run A sits at the card's software power cap, which truncates its honest energy and tightens the bar B must clear. The topmost witness is thin: at $h = 0.216$ B's most expensive repeat stays below the threshold by only 0.86% of the honest mean, so 0.41 is the largest load we could get accepted.

Why the two ends do not meet. The witnesses meet the shared constraints above (i.e. one window, one declaration, one device, etc.) but reach none of the band edges: the achieved rates are $1.84\,\mathrm{pJ/op}$ for the expensive declared arm (band ceiling 2.19), 0.82 for the cheap arm, and 0.54 for the covert fill (marginal edge 0.51). That is why the accepted 0.41 at $h = 0.216$ sits below the model's $\beta(0.216) = 0.72$: specifically, $0.41 \le \beta_{\mathrm{ID}}(0.216) \le 0.72$. This is the relaxation gap of Equation (43), measured rather than assumed. Time is the other reason. The clip Equation (20) converts idle seconds into covert operations at published peak rates, but at the rates these kernels achieve, run B's work no longer fits the window above $h_{\max} \approx 0.24$, while the headline maximiser sits at $h^* = 0.42$; confined to $h \le h_{\max}$, the model bound reads ≈ 0.77. We do not quote 0.77 as a bound, because the ceiling is set by our kernels' achieved rates rather than the adversary's best. The general lesson survives the caveat: a verifier reading time as well as energy constrains the adversary more than the energy channel alone does, and joint energy–time inversion is future work.

C. Benchmark protocol, hardware, and provenance

This appendix is the measurement record: the hardware and provenance of every experiment (including which card measured which band), the workload, how each swept driver is set, and how the energy is metered. It is organised around Experiment 1 (Section 4.2), which reads the total energy quotient E/C off a single A100 by holding the workload fixed and moving one factor of Equation (23) at a time. Experiments 2 and 5 reuse the same protocol with a different swept axis, and Experiment 3 reuses its metering inside fixed-length composite windows (declared burst + covert dose + idle pad). The quotient is the band-edge estimand of Section 4.2—not the marginal rate $r = \partial P/\partial F$ of Equation (2), which Experiments 3 and 4 estimate as regression slopes—and we keep the two apart throughout. Both declared band edges are read at the same saturated rate, so their amortised baselines cancel in the band-edge difference the bound uses; the $\approx 5\%$ spread in achieved rate across cells leaves a small residual, which widens the slack conservatively.

Hardware and provenance. Experiments 1–3 and 5 ran sequentially on one NVIDIA A100-SXM4-80GB (driver 595.84), in a single Slurm allocation (340, 85, 70, and 135 recorded windows, respectively; these totals include the periodic reference-cell windows and the trimmed repeat of each cell, which the figure captions' window counts exclude). Seven cards of one SKU appear in total, by UUID prefix: `GPU-ac1cbae9` (Experiments 1–3 and 5); `GPU-4bb93dfb`, `GPU-617f2717`, `GPU-aa412071` (cross-device sweep); `GPU-757ff02a` (the trained-weight total quotient, reported only as a contrast to the marginal edge); `GPU-747045fa` (Experiment 4, the incremental useful-work edge that prices the bottom two rungs of Section 5); and `GPU-108c1549` (the matched-energy witnesses of Section 4.2). Experiment 4, the trained-weight quotient, and the witnesses are therefore each read against declared edges measured on a different card. The cross-device

sweep bounds the error this introduces: re-measured on three further cards of the same SKU, every band edge agrees to 2–3% (Section 5). Full manifests accompany the released records; the trained weight/activation tensors themselves are not committed, but the manifests record their SHA-256 hashes and the GPU pre-flight validation metrics, so the extraction is checkable against the release but not replayable from the repository alone.

Operating point. The platform denied clock locking and power capping, so DVFS is a recorded axis rather than a controlled one: each window logs the governor-selected SM clock (achieved 1335–1410 MHz on the band sweeps), the bands are stratified into 45 MHz clock strata, and no window was flagged with a *harmful* throttle (thermal or hardware power brake). The cells with the most expensive operands do sit at the card's software power cap by design, which truncates their measured power and is therefore conservative for a most-expensive-rate edge.

Meter accuracy and cadence. `nvmlDeviceGetTotalEnergyConsumption` exposes a cumulative millijoule register integrated in firmware. NVML publishes no accuracy specification for this counter on GA100; the documented $\pm 5\%$ applies to the power readings of the earlier Fermi and Kepler generations (NVIDIA Corporation, 2026), and we adopt that figure as a working meter tolerance wherever one is needed, always in the direction that widens the bound. On the A100, NVML's power readings update every $\sim$100 ms and average only a 25 ms window of each period (Yang et al., 2024). Whether the energy counter shares this sampling is not documented. Each measured cell therefore fills a $\sim$1.5 s window ($\gg$100 ms), as described under *Metering the energy* below.

The workload and its exact operation count. Every cell runs a dense square matrix multiply $A\,B$ with $A \in \mathbb{R}^{M\times K}$, $B \in \mathbb{R}^{K\times N}$ (we take $M = N = K = n$). Each of the MN output entries is a dot product of length K—K multiplies and $K-1$ adds—so by the standard two-operations-per-multiply–add convention the matmul costs

$$C \;=\; 2MNK \;=\; 2n^3 \quad \text{op}, \tag{44}$$

counted as nominal operations of whatever format the cell runs (int8 multiply–adds included; Section 2.1), known analytically from the shapes alone, with no profiler or hardware counter involved. A measured window runs `iters` such matmuls back to back, so its nominal compute is $C_{\text{win}} = 2n^3\,$`iters` and the energy per operation is the quotient $E_{\text{win}}/C_{\text{win}}$. Because C is exact under the nominal convention (the literal scalar count is $MN(2K-1)$, within 0.03% of $2MNK$ at the band cells' $K = 2048$), the quotient inherits the meter's accuracy directly; and because it is intensive (energy *per* operation), cells of different size and `iters` compare directly.

Setting the swept drivers. The bound reads only the extremes of the quotient, so what the sweep needs from each factor of Equation (23) is its range. Every swept axis below is built to drive the factor it controls to both extremes. Each must also move without disturbing the operation count Equation (44), or the quotient E/C would change for two reasons at once.

- **Precision** κ is the tensor `dtype`: `fp32`, `tf32` (fp32 operands routed through the tf32 tensor-core path), `fp16`, `bf16`, and `int8` (an integer-matmul kernel, int8×int8→int32). Lower precision charges less capacitance per multiply-accumulate, lowering κ.
- **Locality** C_{eff} is operand residency at a byte-identical shape, not problem size: sweeping it by changing n would change the operation count and confound the comparison, so instead we fix the shape at $n = 2048$ and vary how many distinct operand sets the window rotates through iteration to iteration. The A100 has a 40 MB L2; at $n = 2048$ an fp16 A–B operand set is $\sim$17 MB, so:
  - *resident* (one operand set, reused every iteration): the working set fits the L2, so most traffic is served on-chip and HBM activation is avoided: low C_{eff};
  - *streamed* (eight operand sets, round-robined): a reuse is evicted before it recurs, so every iteration streams fresh tiles from HBM, engaging its arrays, peripheral logic, and I/O: high C_{eff}.

  Both arms run the identical shape, kernel, and operation count Equation (44); only the memory-traffic path differs, so locality no longer confounds the count. The kernel stays compute-bound in both arms (a 2048^3 matmul is high arithmetic intensity) so streaming raises C_{eff} without making the workload memory-bound, and Figure 5 accordingly shows locality moves r only slightly. The residency margin is comfortable only for the two-byte formats. An fp32 or tf32 operand set stores four bytes per element, so it is twice the size, $\sim$34 MB against the 40 MB L2: its resident arm marginally spills, and the locality contrast we measure for those two formats is attenuated accordingly.

The third driver, supply voltage V^2 (DVFS), we could not sweep: the platform denied user clock locking, so the clock was left to the governor and merely recorded per run. The operand *values* are a fourth axis: Experiment 1 crosses all six operand distributions with precision and locality in a full factorial, so the pooled envelope already contains the operand spread; Experiment 5 re-runs the operand axis alone, at fixed precision and locality, as the descriptive operand-only map of Appendix D.

Metering the energy. Energy is read from the on-die NVML total-energy counter: the window energy is one subtraction $E_{\text{end}} - E_{\text{start}}$, with a `cuda.synchronize()` fence bracketing each latch so the delta spans exactly the window's kernels. (The harness falls back to integrating the power query in a background sampler where the counter is unavailable; no recorded window used the fallback.) Each cell fills a $\sim$1.5 s window with matmuls run back to back on pre-allocated buffers, with no host traffic inside the window: long enough that counter quantisation is negligible, and saturated enough that the joules are the datapath doing arithmetic, not allocation or transfer. A few warm-up windows absorb JIT and clock ramp, several measured windows follow, and the per-cell median is kept; shuffling the cell order and periodically re-measuring a fixed reference cell guards against slow thermal drift. The largest per-cell median over the sweep is $r_{\text{hi}}^{\text{dec}}$ with no format pinned, and the largest within one format is that format's own ceiling; Figure 5 plots the full set.

D. How Experiment 5 varies the operands

This appendix documents Experiment 5, the operand-only sweep: what it measures and how each of its six operand distributions is constructed. The spread it measures, $w_0 = 1.83$, independently supports Experiment 1's operand width $w_\alpha = 1.88$, although no security bound rests on any number in this appendix. The sweep also resolves the activity factor into named distributions, which is what the threat-model rungs of Section 5 act on when they exclude zero operands.

Experiment 5 holds the kernel, shape, and precision (fp16) fixed and moves only the operand values. Every configuration runs the identical matmul $A\,B$ with the identical operation count Equation (44), so the energy differences track the switching-activity factor α of Equation (23)—how many bits physically toggle in the datapath as the multiply–accumulates stream through. A datapath that sees the same bits over and over charges little capacitance; one whose bits flip on every operation charges the most. The six distributions are constructed to walk α from its floor to its ceiling; they are the same six families Experiment 1 crosses with precision and locality in its factorial (Appendix C), so Experiment 1's fp16 cells contain this comparison as one slice of a larger grid.

The measurement. Operand values alone spread r by an observed factor $w_0 = 1.83$ (Figure 9), following the textbook ordering zeros $<$ low-entropy $<$ structured-sparse $<$ dense-random $\approx$ declared-typical $\approx$ adversarial-toggle. The clock confounds the comparison. Locking is privilege-denied (Section 4.2), so the families did not run at a common clock: the governor gave the cheapest (zeros, low-entropy) the highest clocks and the most expensive the lowest, a monotone $1410 \to 1335$ MHz slide against operand cost. A faster clock carries a higher supply voltage, and the voltage sets the energy per operation (Equation (39)), so the slide squeezed the spread from both ends: the cheap families paid a voltage premium that raised the floor, and the expensive families ran at a voltage discount that lowered the ceiling. At one common clock both effects disappear and the spread widens. The prediction is therefore directional: $w_0 = 1.83$ should sit below a matched-clock measurement of the same factor. Experiment 1 is such a measurement. Its operand families share a median 1410 MHz and its width $w_\alpha = 1.88$ lies above w_0, on the predicted side, agreeing to 3%. That 1.88 is a median over ten format–locality groups, only one of which runs what this sweep runs. Restricting to that group, i.e. Experiment 1's fp16 streamed cells, gives 1.94: again above w_0, again on the side the confound predicts.

The distributions. Below we show each on a 4×4 patch; the real operands are the same patterns at full size.

1. Zeros—the activity floor. Every entry is zero, so no bits ever change and α is at its minimum:

$$A_{\text{zeros}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

This is the cheapest run at fixed fp16 and sets the descriptive fp16 operand floor, 1.16 pJ/op as a total quotient; combined with the int8 format in Experiment 1's factorial it reaches the cheapest measured quotient, 0.786 pJ/op, which is $r_{\text{lo}}^{\text{dec}}$ with

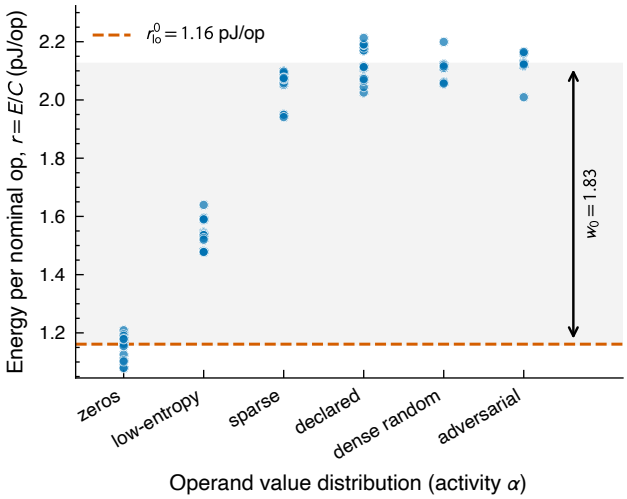


Figure 9. The shape of the activity factor α (Experiment 5, $n = 96$ windows across six operand distributions). Kernel, shape, and precision (fp16) are held fixed and only the operand values vary; the clock was governor-selected, not locked, and slides monotonically against operand cost, so this map is descriptive. The shaded span is the observed spread $w_0 = 1.83$, just below Experiment 1's matched-clock $w_\alpha = 1.88$, on the side the confound predicts. The minimum (zeros, dashed) is the descriptive fp16 operand floor 1.16 pJ/op as a total quotient—*not* the covert denominator, which is Experiment 3's marginal edge.

no format pinned. (Both are declared entries. The covert entry $r_{\text{lo}}^{\text{cov}}$ is Experiment 3's marginal cost, not either of these quotients; Section 4.2 and Table 1.)

2. Low-entropy—a few distinct levels. Entries are drawn from only four values $\{0, 1, 2, 3\}$. With so few distinct bit patterns, few bits differ between neighbouring operands, so operand-bit entropy, and hence switching, stays low:

$$A_{\text{low-entropy}} = \begin{pmatrix} 1 & 0 & 3 & 1 \\ 2 & 2 & 0 & 3 \\ 0 & 1 & 1 & 2 \\ 3 & 0 & 2 & 1 \end{pmatrix}.$$

3. Structured-sparse—half the operands forced to zero. Dense random values, but a fixed half of the columns are deterministically zeroed (every other column), giving exactly 50% zeros in a regular pattern: the structured sparsity real accelerators exploit, rather than random dropout. The persistent zero columns roughly halve the switching relative to a fully dense operand:

$$A_{\text{sparse}} = \begin{pmatrix} 0 & 0.4 & 0 & -0.9 \\ 0 & -0.7 & 0 & 0.2 \\ 0 & 0.8 & 0 & 0.5 \\ 0 & -0.3 & 0 & -0.6 \end{pmatrix}.$$

4. Dense-random—generic busy data. Every entry is an independent uniform draw from $[-1, 1]$: no zeros, full-width mantissas, high but not maximal switching. This is the realistic-workload reference:

$$A_{\text{dense}} = \begin{pmatrix} 0.31 & -0.82 & 0.55 & -0.14 \\ -0.67 & 0.09 & -0.48 & 0.73 \\ 0.21 & 0.64 & -0.95 & 0.38 \\ -0.52 & -0.27 & 0.86 & -0.41 \end{pmatrix}.$$

5. Declared-typical—the realistic-activation stand-in. Every entry is an independent standard-normal draw, $\mathcal{N}(0, 1)$: a proxy for the activations and weights a genuine declared workload streams, with full-width mantissas and no engineered structure. Its switching sits alongside dense-random (i.e. high but not maximal) and it is the operand class that characterises

the honest-window energy scatter used by the declared-work model:

$$A_{\text{decl}} = \begin{pmatrix} 0.42 & -1.13 & 0.27 & -0.68 \\ -0.35 & 0.91 & -1.42 & 0.11 \\ 1.06 & -0.24 & 0.58 & -0.83 \\ -0.72 & 0.39 & -0.16 & 1.24 \end{pmatrix}.$$

6. Adversarial-toggle—the activity ceiling. Engineered for maximal switching: the matrix is a checkerboard of two complementary bit patterns, `0x5555...` and its bitwise inverse `0xAAAA...`, so adjacent operands differ in every bit (maximal Hamming distance) and the operand path sees maximal input-level switching per operation. For floats the pattern is built in an integer bit-view of the same width and reinterpreted as the float type, so the raw bits are controlled exactly. Writing the two patterns as p and $\bar{p}$:

$$A_{\text{adv}} = \begin{pmatrix} p & \bar{p} & p & \bar{p} \\ \bar{p} & p & \bar{p} & p \\ p & \bar{p} & p & \bar{p} \\ \bar{p} & p & \bar{p} & p \end{pmatrix}, \qquad p = \texttt{0x5555...},\ \bar{p} = \texttt{0xAAAA...}$$

A matmul forms each output as a dot product $\text{out}[i,j] = \sum_k A[i,k]\,B[k,j]$, streaming $k = 0, 1, 2, \ldots$ and loading one operand into the multiplier's input register each cycle. Because every row and every column of the checkerboard alternates $p, \bar{p}, p, \ldots$, that register walks

$$\underbrace{\texttt{0101 0101}}_{k=0} \rightarrow \underbrace{\texttt{1010 1010}}_{k=1} \rightarrow \underbrace{\texttt{0101 0101}}_{k=2} \rightarrow \cdots$$

flipping all its bits on every cycle. Dynamic CMOS energy is paid per bit-transition, so this maximises the switching the operand registers can drive: a w-bit operand toggles w bits per cycle, against $\approx w/2$ for dense-random (two random words differ in about half their bits) and 0 for zeros or any constant operand (the register never changes). The checkerboard is simply the 2-D layout that makes the row sequence (feeding A) and the column sequence (feeding B) both alternate this way at once, for every output entry.

The energy-per-operation spread from the cheapest patch (A_{zeros}) to the densest operands (dense-random, declared-typical, and the adversarial checkerboard), at fixed operation count, is the observed operand spread w_0 of Figure 9. Per the clock confound above, the spread is descriptive within-device evidence, not a certified population edge—and maximal input-level toggling does not itself certify maximal switching inside the tensor-core internals, whose operand encodings are opaque (nor do zero operands mean no hardware bits switch); no security bound in the paper rests on it.